第8章 查找

预备知识

8.1 静态查找表

8.2 动态查找表

8.3 哈希表

本章重点难点

重点：顺序查找、二分查找、索引顺序查找、二叉排序树查找以及散列表查找的基本思想和算法实现。

难点：二叉排序树的删除算法和平衡二叉树的构造算法。

第8章 查找

预备知识

8.1 静态查找表

8.2 动态查找表

8.3 哈希表

1. 有关概念
2. 类型定义
3. 宏定义

预备知识

1. 有关概念

(1) 查找表：

是由同一类型的数据元素(或记录)构成的集合。

(2) 静态查找表：

仅作查询和检索操作的查找表。

(3) 动态查找表：

在查找时包含插入、删除或修改。

预备知识

1. 有关概念

	姓名	年龄	性别	成绩
	张三	20	男	86
	李四	19	男	85
	……	……	……	……

(4) 主关键字

可以唯一的识别一个记录的数据项（字段）。

(5) 次关键字

关联若干项记录的数据项（字段）。

(6) 查找

根据给定的某个值，在查找表中确定一个其关键字等于给定值的数据元素（记录）。

1. 有关概念

(7) 查找成功

查找表中存在满足条件的记录。

(8) 查找不成功

查找表中不存在满足条件的记录。

2. 类型定义

```
typedef float Keytype;
//Keytype定义为浮点数据类型
```

或者

```
typedef int Keytype;
//Keytype定义为整型数据类型
```

或者

```
typedef char *Keytype;
//Keytype定义为字符指针数据类型
```

2. 类型定义

数据元素类型定义为：

```
typedef struct{
    Keytype key;//关键字
    …
}ElemType;
// ElemType结构体数据类型
```

学号	姓名	年龄	性别	成绩
1001	张三	20	男	86
1002	李四	19	男	85
1003	……	……	……	……

查找的性能

- 查找算法时间性能由关键字的比较次数来度量。
- 同一查找集合、同一查找算法，关键字的比较次数与哪些因素有关呢？
- 查找算法的时间复杂度是问题规模n和待查关键字在查找集合中的位置k的函数，记为$T(n, k)$。

查找的性能

- 平均查找长度：
  - 把给定值与关键字进行比较的次数的期望值称为查找算法在查找成功时的平均查找长度--ASL(Average Search Length)。
  - 计算公式：假设查找集合中的记录个数p_i为查找表中第i个记录的概率，$\sum p_i = 1$，c_i为查找第i个记录所进行的比较次数，则

$$ASL = \sum_{i=1}^{n} p_i c_i$$

  - 在等概率情况下，即$p_i = 1/n$时，

$$ASL = \frac{1}{n} \sum_{i=1}^{n} c_i$$

查找的分类：

1. 基于关键字比较的查找
   - 顺序查找、折半查找、分块查找、BST&AVL、B-树和B^+树
2. 基于关键字存储位置的查找
   - 散列法

- 根据被查找的数据集合存储位置
  1. 内查找：整个查找过程都在内存进行；
  2. 外查找：若查找过程中需要访问外存。

第8章 查找

预备知识

8.1 静态查找表

8.2 动态查找表

8.3 哈希表

8.1.1 顺序表的查找

8.1.2 有序表的查找

*8.1.3 静态树表的查找

8.1.4 索引顺序表的查找

第8章 查找

8.1.1 顺序表的查找

8.1.2 有序表的查找

*8.1.3 静态树表的查找

8.1.4 索引顺序表的查找

1. 存储结构
2. 查找思想
3. 实例演示
4. 算法实现
5. 算法分析

□ 静态查找表的抽象数据类型

ADT StaticSearchTable {

数据对象D： D是具有相同特性的数据元素的集合。每个数据元素含有类型相同的关键字，可唯一标识数据元素。

数据关系R： 数据元素同属一个集合。

基本操作 P： 见下页

} ADT StaticSearchTable

基本操作

Create(&ST, n);　//建立静态查找表

Destroy(&ST);　//销毁静态查找表

Search(ST, key); //按关键字字key查找

Traverse(ST, Visit()); //遍历查找表

Insert操作的实现
Delete操作的实现
} 不适合顺序表

1. 顺序查找表存储结构

```
typedef struct {
    ElemType *elem;
 // 数据元素存储空间基址，建表时 按实际长度分配，0号单元留空
    int     length;     // 表的长度
} SSTable;
```

```
SSTable        ST;
ST.elem[i].key
```

```
typedef struct{
    Keytype key;//关键字
    …
}ElemType;
```

key	name	age	sex	score
1001	张三	20	男	86
1002	李四	19	男	85
1003	……	……	……	……

2. 顺序查找的思想

(1)从查找表的第一个元素向后（或从最后一个元素向前），比较当前位置数据元素的关键字与查找关键字；

(2)若相等，输出当前位置，查找成功，若不相等，走向下一个位置；

(3)循环执行(1)、(2)步，直到查找成功或超出范围，表示查找失败。

3. 顺序查找表示例演示

查找55，从后向前

	67	78	76	45	33	99	88

监视哨

55	67	78	76	45	33	99	88

表示没找到，返回值为0

问题：监视哨的作用是什么?

在顺序查找时省略了下标的越界检查，提高了效率。

3. 顺序查找表示例演示

查找33，从后向前

	67	78	76	45	33	99	88

监视哨

33	67	78	76	45	33	99	88

查找成功，返回当前位置5

4. 顺序查找表的算法与实现

```
int Search_Seq(SSTable ST, KeyType key) {
            // 在顺序表ST中顺序查找其关键字等于
            //  key的数据元素。若找到，则函数值
            // 为该元素在表中的位置，否则为0。
  ST.elem[0].key = key;
                                        // “哨兵”
  for (i=ST.length; ST.elem[i].key!=key;  --i);
                                        // 从后往前找
  return i;
                                        // 找不到时，i为0
} // Search_Seq
```

5. 顺序查找表算法分析

进行比较的关键字个数的期望值:

$$ASL = \sum_{i=1}^{n} P_i C_i$$

查找成功时的平均查找长度

n **为表长**

P_i **为查找表中第i个记录被查找的概率**

C_i**为找到记录i时，关键字的比较次数**

5. 顺序查找表算法分析

在等概率查找的情况下，

$$P_i = \frac{1}{n}$$

顺序表查找的平均查找长度为：

$$ASL_{ss} = \frac{1}{n}\sum_{i=1}^{n}(n-i+1) = \frac{n+1}{2}$$

思考题：在概率不等的情况下，*ASL*在什么情况下取最小值?

5. 顺序查找表算法分析

答案：在不等概率查找的情况下，*ASL* 在

$$P_n \geq P_{n-1} \geq \cdots \geq P_2 \geq P_1$$

时取最小值。

$$ASL_{失败}=n+1$$

单选题 1分

设置

从一个具有 n 个结点的单链表中查找其值等于 x 的结点时，在查找成功的情况下，需平均比较（ ） 个结点。

A n

B n/2

C (n-1)/2

D (n+1)/2

提交

第8章 查找

8.1.1 顺序表的查找

8.1.2 有序表的查找

*8.1.3 静态树表的查找

8.1.4 索引顺序表的查找

1. 折半查找的存储要求
2. 折半查找的思想
3. 实例演示
4. 算法实现
5. 算法分析

1. 折半查找的前提要求

想一想：英文字典的特点及英语单词的查找过程。

英文字典是有序的，英语单词的查找类似于折半查找。

折半查找的前提要求是顺序存储并且有序。

2. 折半查找表的思想

(1)、将要查找的关键字与查找表中间的元素的关键字进行比较，若相等，返回当前位置；

(2)、若查找关键字比当前位置关键字小，向前递归，否则向后递归。

3. 折半查找表实例演示

查找21

位置	0	1	2	3	4	5	6	7	8	9	10
关键字	05	13	19	21	37	56	64	75	80	88	90

low

mid

mid= (low+high)/2

high

high=mid-1

low=mid+1

mid

分析查找22

4. 折半查找表算法与实现

```
int Search_Bin ( SSTable ST, KeyType key ) {
  low = 1;  high = ST.length;          // 置区间初值
  while (low <= high)
  {
      ..................
  }
  return 0;                  // 顺序表中不存在待查元素
} // Search_Bin
```

4. 折半查找表算法与实现

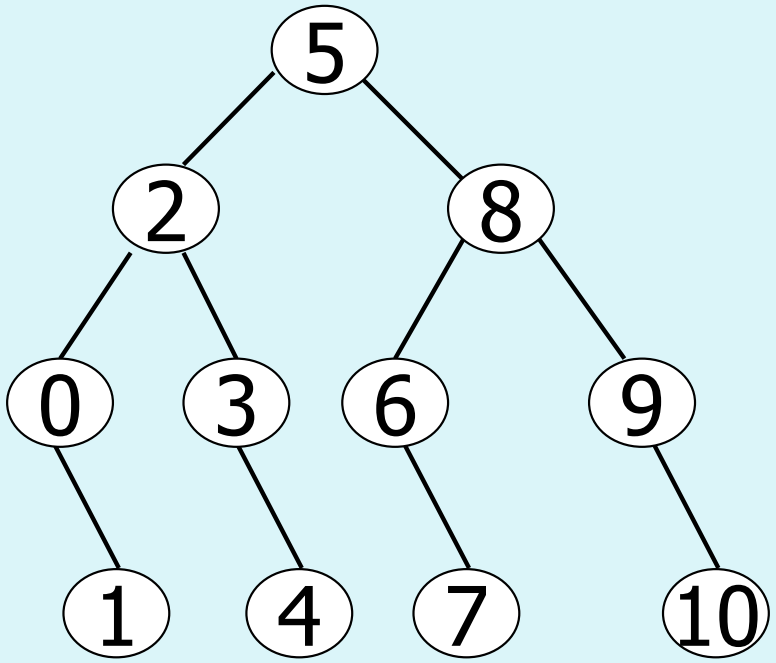
```
while (low <= high) {
    mid = (low + high) / 2;
    if  (EQ (key , ST.elem[mid].key) )
      return  mid;                          // 找到待查元素
    else  if ( LT (key , ST.elem[mid].key) )
      high = mid - 1;        // 继续在前半区间进行查找
    else  low = mid + 1; // 继续在后半区间进行查找
}
```

注意：折半查找只适合于静态查找！

5. 折半查找表算法分析

0	1	2	3	4	5	6	7	8	9	10
05	13	19	21	37	56	64	75	80	88	90

用判定树描述折半查找的过程。



$$O(\log_2 n)$$

对于n个元素，二分查找，成功时最多的关键字比较次数为：$\lceil \log_2(n+1) \rceil$
不成功时关键字比较次数为：$\lceil \log_2(n+1) \rceil$。

折半查找的递归算法实现

```
int  BinSearch2(SSTable ST, int low, int high, , KeyType key )
{
   if (low>high) return 0;
   else {
      mid=(low+high)/2;
      if (key < ST[mid].key )
            return BinSearch2(ST, low, mid-1, key);
      else  if (key>ST[mid].key)
            return BinSearch2(ST, mid+1, high, key);
      else return mid;
    }
} /*  F必须是顺序有序表(此处为增序);时间复杂度 :O(log2 n)  */
```

单选题 1分

设置

设有100 个元素的有序表，采用折半查找方法，成功时最大的比较次数是（ ）。

A 25

B 50

C 10

D 7

提交

第8章 查找

8.1.1 顺序表的查找

8.1.2 有序表的查找

*8.1.3 静态树表的查找

8.1.4 索引顺序表的查找

1. 索引顺序表的存储要求
2. 实例演示
3. 查找思想
4. 存储结构
5. 算法实现
6. 算法分析

1. 索引顺序表的存储要求

索引表

该块最大关键字	22	48	86
该块起始地址	0	4	7

22	12	9	20	33	42	48	49	60	86	53
0	1	2	3	4	5	6	7	8	9	10

线性表

1. 索引顺序表的存储要求

■ **线性表的要求**

查找表分成n块，当i>j时，第i块中的最小元素的关键字大于第j块中的最大元素的关键字。

■ **索引表的要求**

(1) 索引表是顺序存储

(2) 索引表里存储了各个块最大值和开始地址。

2. 索引顺序表的查找示例演示

22	48	86
0	4	7

22	12	9	20	33	42	48	49	60	86	53
0	1	2	3	4	5	6	7	8	9	10

分别讨论查找12、42、41、60、89

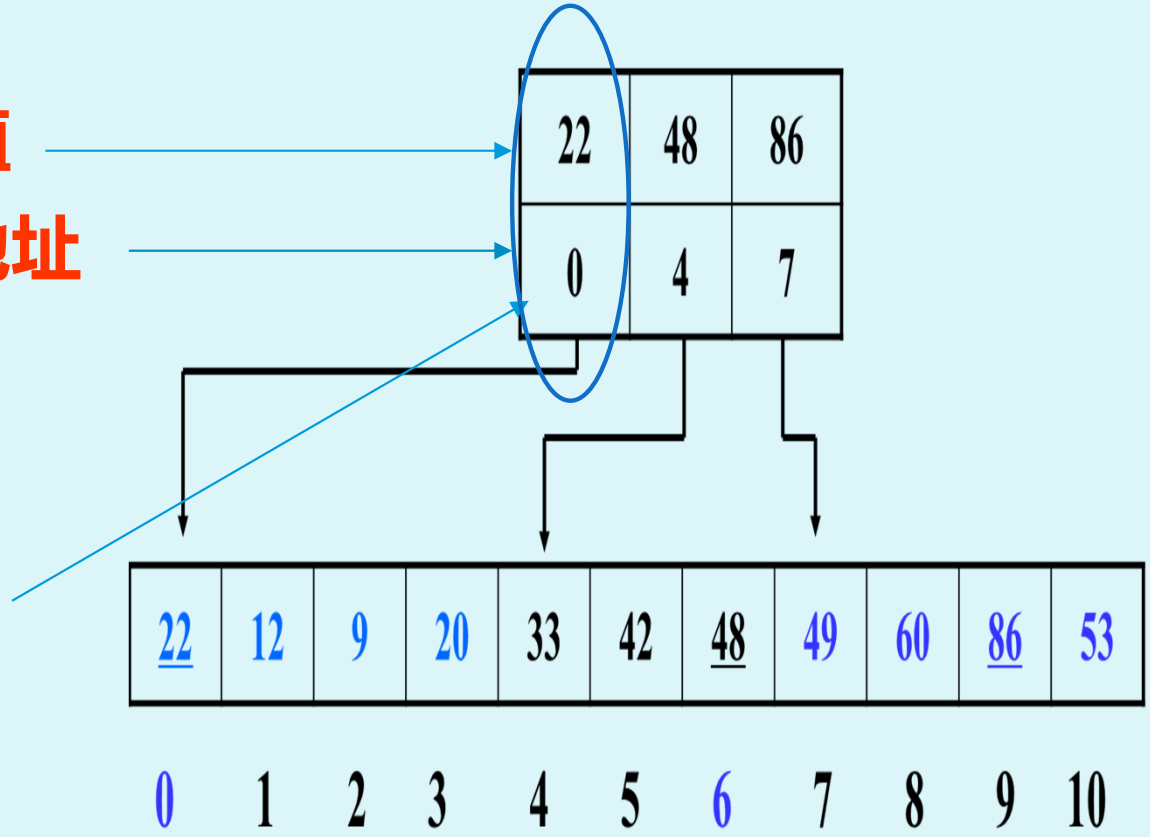
3. 索引顺序表的查找思想

(1) 首先确定所要查找关键字在哪一块中;

(2) 在所确定的块中用顺序查找查找关键字。

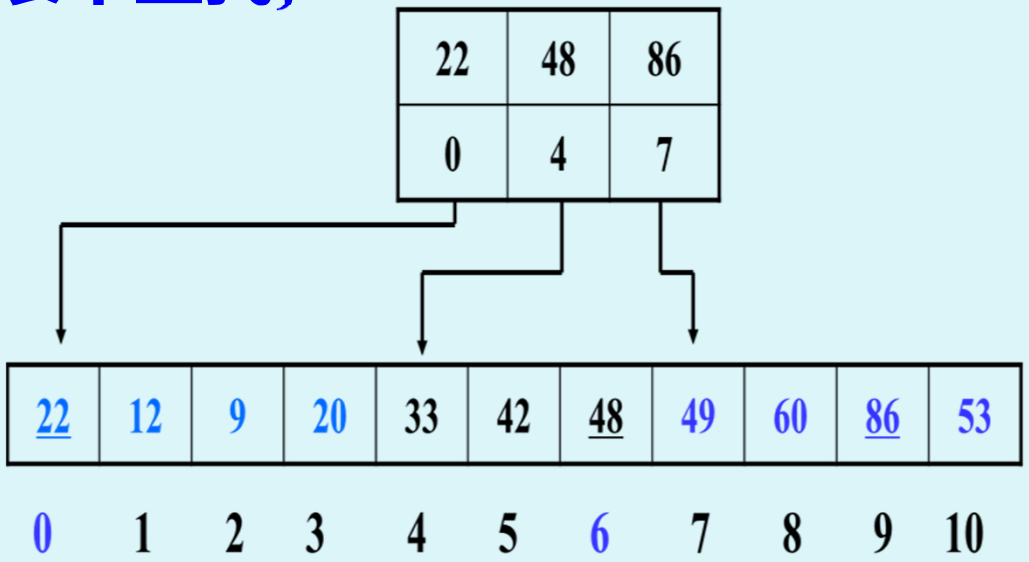
4. 索引表的存储结构

```
typedef struct
{keytype key;      //本块最大值
 int addr;         //本块开始地址
}indextype;
typedef struct
{indextype index[maxblock];
 int block;
}INtable;
INtable IX;
```

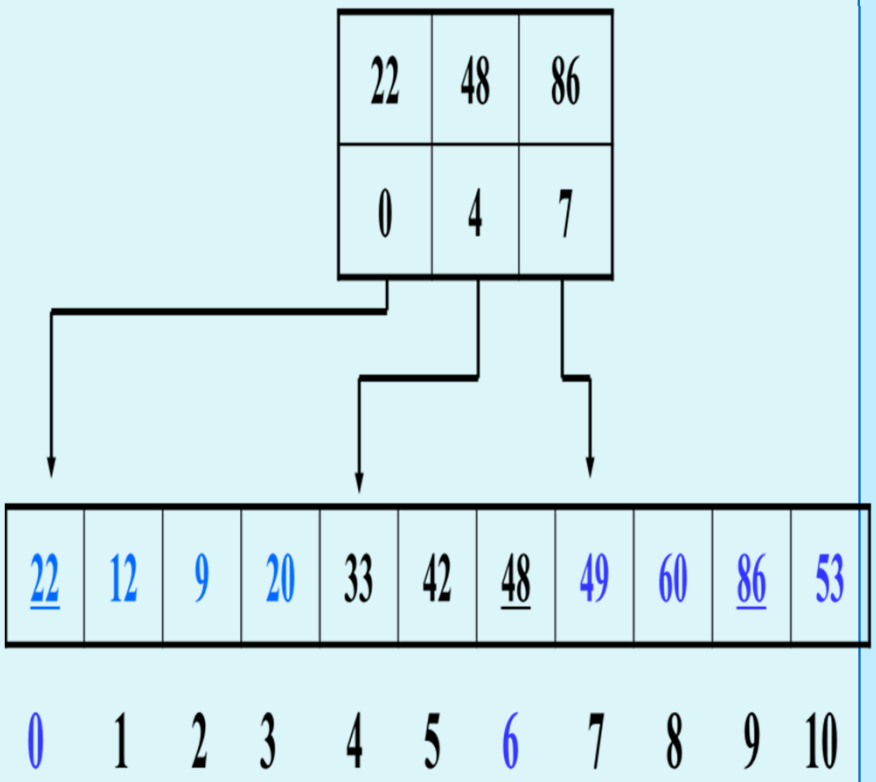


5. 索引顺序表的算法与实现

```
int SEARCH(SSTable ST, INtable IX, KeyType key)
{
  int i=0,s,e;
                    //s记录在查找表中的开始位置
                    //e记录在查找表中的结束位置
  {在索引表中查找，确定s和e的值}
  {根据s和e的值在线性表中查找}
   return -1
}
```



5. 索引顺序表的算法与实现




```
while ((key > IX.index[i].key)&&(i < IX.block))
        i++;

if (i < IX.block)
{ s = IX.index[i].addr;
  if (i == IX.block-1)   e = ST.length;
  else e = IX.index[i+1].addr-1;

while( key!=ST.elem[s].key&&(s<=e) )
    s=s+1;
  if (s<=e)   return s;
}
```

6. 索引顺序表的算法分析

问题：如果索引表长度为b，每块平均长度为L平均查找长度是多少?

答案：(b+1)/2+(L+1)/2。

问题：长度为n的线性表,平均分成多少块平均查找次数最少?

答案：

$$\sqrt{n}$$

索引顺序表的查找——分块查找

- 分块查找局限性和改进
  - 只适合静态查找；
  - 改进：
    - 在索引表中保存各块的下标范围，此时不必均匀分块
    - 各块存放在不同的向量（一维数组）中；
    - 把同一块中的元素组织成一个链表。
- 分块查找的优点
- ①在表中插入或删除一个记录时，只要找到该记录所属的块，就在该块内进行插入和删除运算。因块内记录的存放是任意的，所以插入或删除比较容易，无须移动大量记录。
- 分块查找的主要代价是增加一个辅助数组的存储空间和将初始表分块排序的运算。

第8章 查找

预备知识

8.1 静态查找表

8.2 动态查找表

8.3 哈希表

8.2.1 二叉排序树和平衡二叉树

8.2.2 B树和B+树

//自学

8.2.1 二叉排序树和平衡二叉树

- 二叉排序树的定义
- 二叉排序树的结点查找
- 二叉排序树的结点插入
- 二叉排序树的结点删除
- 平衡二叉树的构建

■ 二叉排序树的定义

二叉排序树(Binary Sort Tree)或者是一棵空树；

或者是具有下列性质的二叉树：

若根节点有左子树，则左子树上所有结点关键字的值均小于根结点关键字的值；

若根节点有右子树，则右子树上所有结点关键字的值均大于根结点的关键字的值；

根节点的左、右子树也分别为二叉排序树。

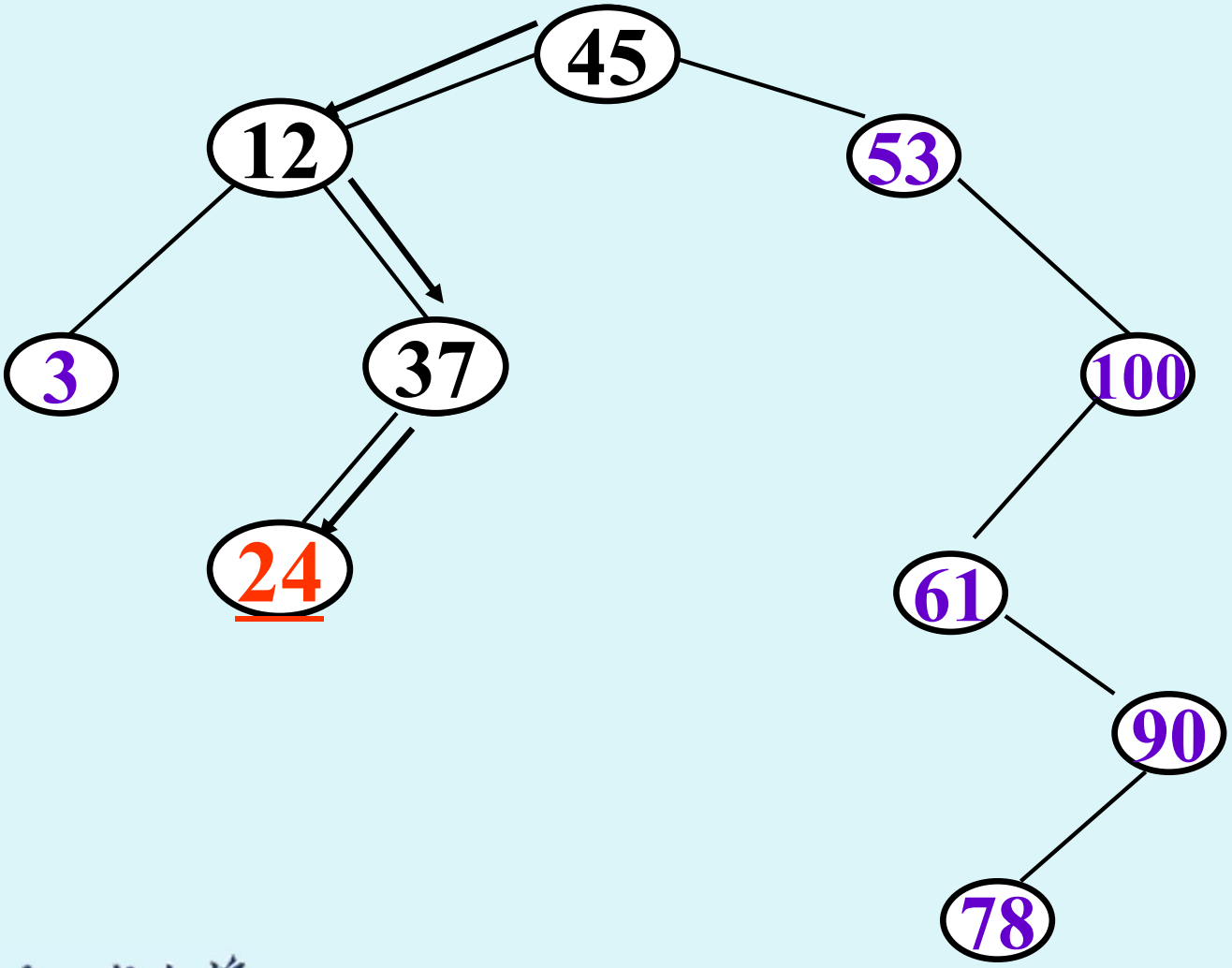
8.2.1 二叉排序树和平衡二叉树

- 二叉排序树的定义
- 二叉排序树的结点查找
- 二叉排序树的结点插入
- 二叉排序树的结点删除
- 平衡二叉树的构建

1. 二叉排序树的查找实例
2. 二叉排序树的查找思想
3. 二叉排序树的存储结构
4. 二叉排序树的查找算法
5. 二叉排序树的查找分析

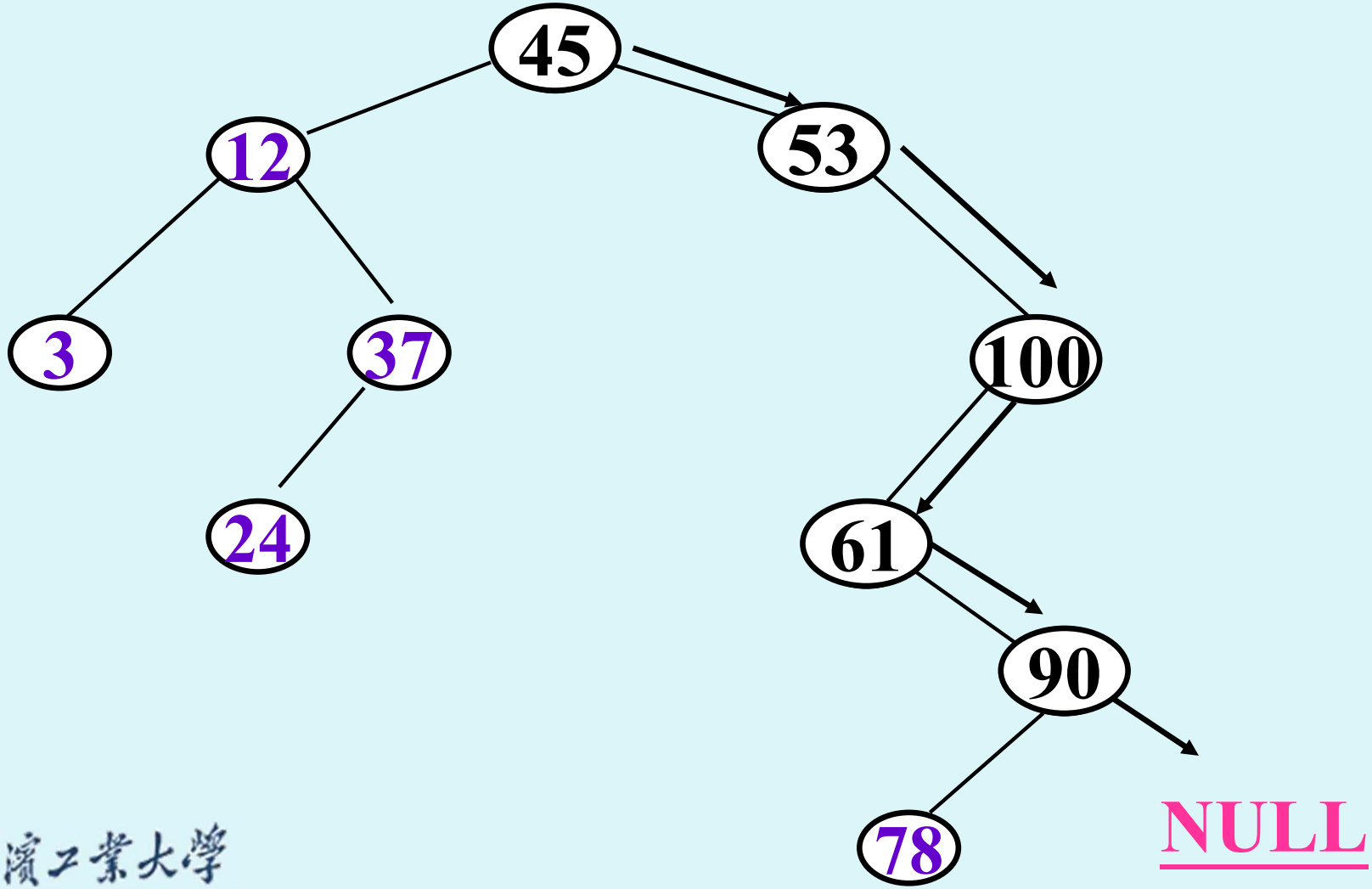
1. 二叉排序树的查找示例演示

例 在下图中查24



1. 二叉排序树的查找示例演示

例 在下图中查95



2. 二叉排序树的查找思想

(1) 当二叉排序树不空时，先将给定值和根结点的关键字比较，若相等，则查找成功；否则：

(2) 若给定值小于根结点的关键字，则在左子树上继续进行查找；

(3) 若给定值大于根结点的关键字，则在右子树上继续进行查找;

(4) 直到找到或查到空结点时为止。

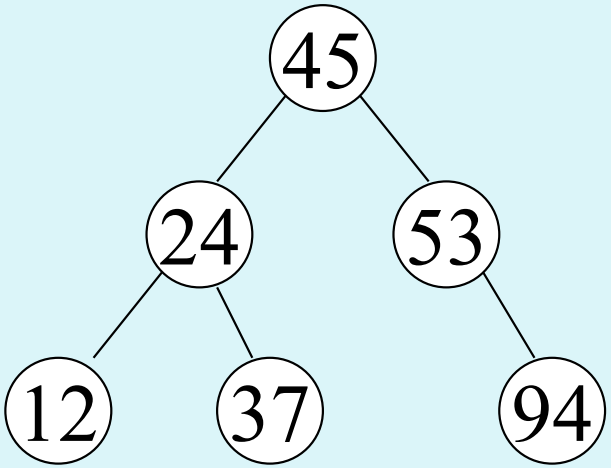
3. 二叉排序树的存储结构

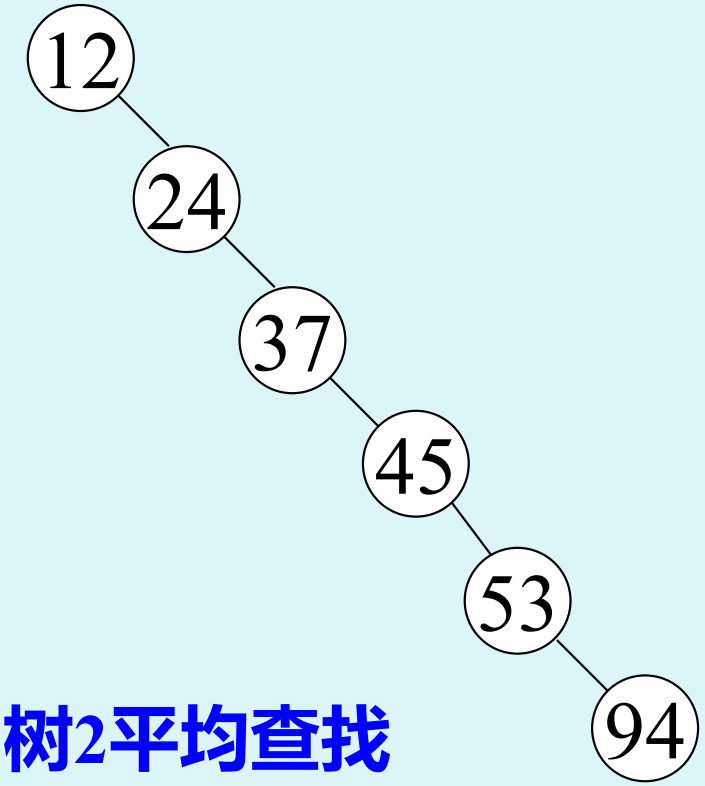
```
typedef struct BiTNode { // 结点结构
    TElemType      data;
    struct BiTNode  *lchild, *rchild;
                                // 左右孩子指针
} BiTNode, *BiTree;
```

4. 二叉排序树的查找算法

```
BiTree SearchBST(BiTree T, keytype k)
 {BiTree p=T;
  while(p!=NULL)
   { if (p->data.key==k)
        return p;
     if (k<p->data.key)
        p=p->lchild;
     else p=p->rchild;
    }
     return NULL;
}
```

5. 二叉排序树的查找算法分析





树1平均查找长度

$O(\log_2 n)$

树2平均查找长度

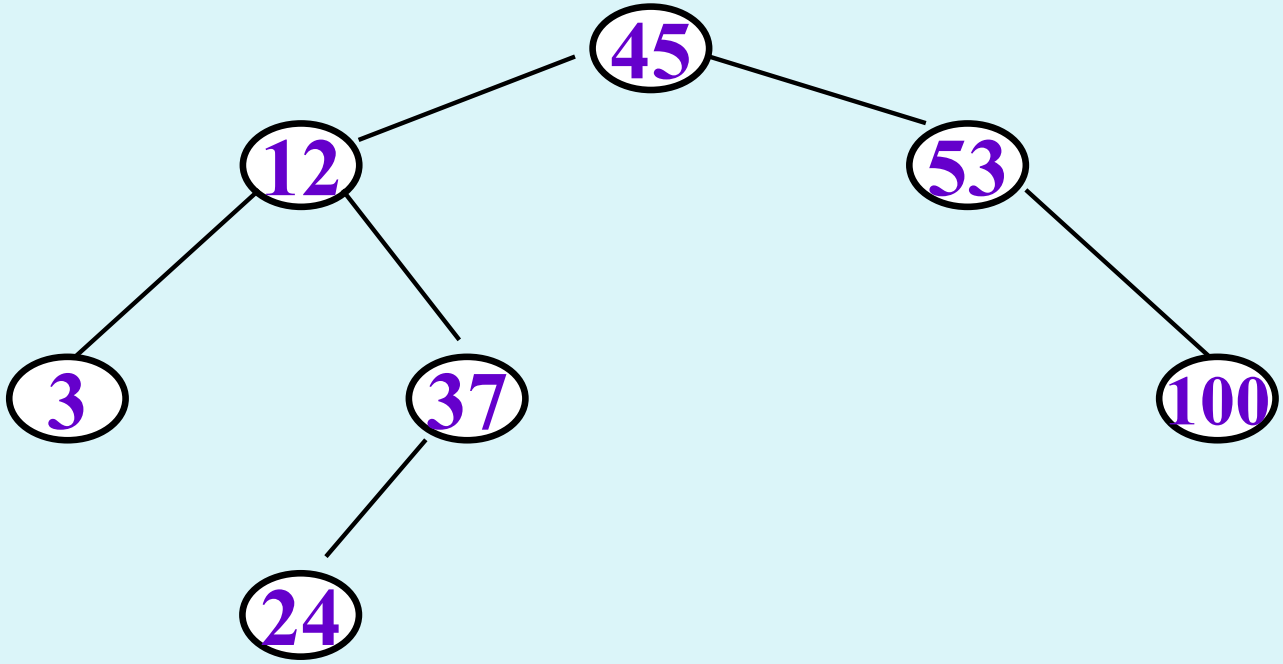
$O(n)$

5. 二叉排序树的查找算法分析

(1) 二叉排序树的平均查找长度最差情况与顺序表相同(关键字有序时),为O(n);

(2) 最好情况与折半查找相同,是O（$\log_2 n$）数量级的;

(3) 二叉排序树的平均查找长度仍然是O($\log_2 n$)。



问题： 若按中序对二叉查找树进行遍历,遍历序列有什么特点?

答案： 按关键字升序排列。

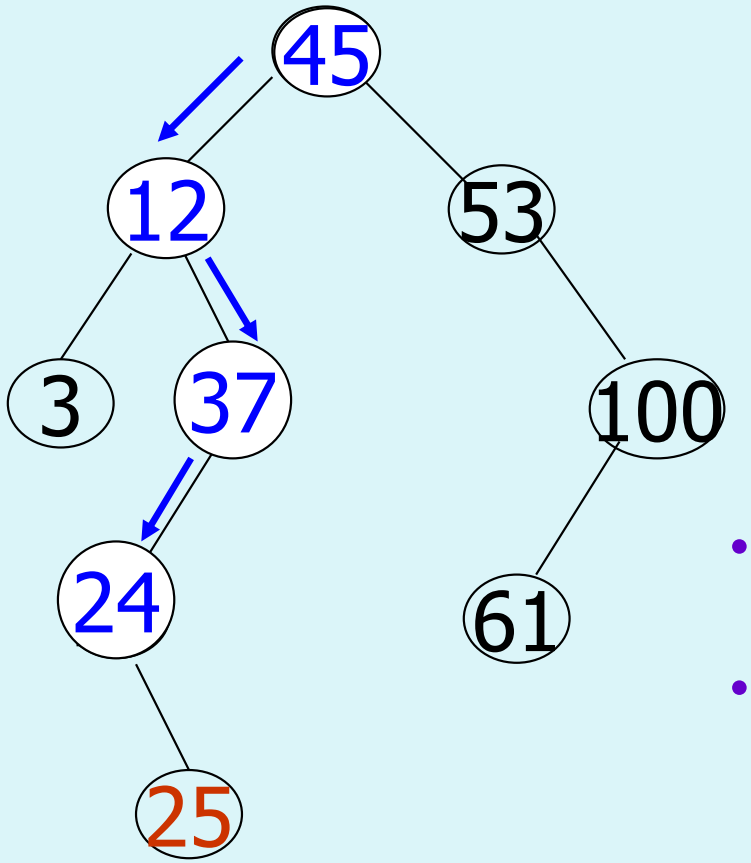
8.2.1 二叉排序树和平衡二叉树

- 二叉排序树的定义
- 二叉排序树的结点查找
- 二叉排序树的结点插入
- 二叉排序树的结点删除
- 平衡二叉树的构建

1. 二叉排序树的插入算法实例
2. 二叉排序树的插入算法思想
3. 二叉排序树插入位置查找
4. 二叉排序树的插入算法
5. 二叉排序树的插入算法分析

1. 二叉排序树的插入过程演示

在如下二叉排序树中插入25过程演示



二叉排序树的插入算法

两种情况：

（1）树为空。

（2）树不空

- 若二叉排序树为空树，则新插入的结点为根结点；
- 否则，新插入的结点必为一个新的叶结点.

2. 二叉排序树的插入算法思想

(1) 若二叉树为空：

则待插入结点s作为根结点；

(2) 当二叉排序树非空时：

将待插结点关键字与根结点进行比较，若相等，则说明树中已有此结点，无须插入；

若小于根结点，插入左子树；

若大于根结点，插入右子树。

3. 二叉排序树的查找算法

为插入算法做准备：完成3个任务(1)记录树是否空？ (2)记录是否查找到相同关键字； (3)若没有查找到，记录查找路径上最后一个节点

```
Status SearchBST(BiTree T, keytype key, BiTree f, BiTree &p)
                                        //f的初值为空
 {if(!T)
        {p=f; return FALSE;} //树为空时，p=NULL,返回值为假值
   else if EQ(key,T->data.key)
        {p=T; return TRUE;} //找到时，p=找到节点,返回真值
   else if LT(key,T->data.key)
        return SearchBST(T->lchild, key, T, p);
   else
        return SearchBST(T->lchild, key, T, p);
 }
```

3. 二叉排序树的查找算法

```
Status SearchBST(BiTree T,keytype key, BiTree f, BiTree &p)
                                        //f的初值为空
 {if(!T)
        {p=f; return FALSE;}
   else if EQ(key,T->data.key)
        {p=T; return TRUE;}
   else if LT(key,T->data.key)
        return SearchBST(T->lchild,  key, T, p);
   ……
```

函数运行后有三种结果：

(1) 返回真值表示找到，无需插入。

(2) 返回假值，p==NULL, 表示是空树。

(3) 返回假值，p!=NULL，插入P的左子树或右子树

4. 二叉排序树的插入算法

```
Status InsertBST(BiTree &T, ElemType e)
 { BiTree p,s;
   if (!SearchBST(T, e.key, NULL, p))                    // 查找不成功
  { s = (BiTree)malloc(sizeof(BiTNode));
    s->data = e;  s->lchild = s->rchild = NULL;
    if (!p) T = s;                                       // 插入 s 为新的根结点
      else if (LT(e.key, p->data.key)) p->lchild=s; // 插入s为左孩子
            else p->rchild = s;                          // 插入 s 为右孩子
    return TRUE;
  }
  else
     return FALSE;    // 树中已有关键字相同的结点，不再插入
} // Insert BST
```

5. 二叉排序树的插入算法分析

(1)二叉排序树的插入算法的时间复杂性与查找算法的时间复杂性相同;

(2)最好情况是O（$\log_2 n$）;最坏情况是O(n);平均情况是O($\log_2 n$)。

问题：如何生成二叉排序树?

答案：从空树开始循环调用插入算法。

8.2.1 二叉排序树和平衡二叉树

- 二叉排序树的定义
- 二叉排序树的结点查找
- 二叉排序树的结点插入
- 二叉排序树的结点删除
- 平衡二叉树的构建

1. 二叉排序树删除结点定义
2. 二叉排序树删除结点方法
3. 二叉排序树删除结点算法
4. 二叉排序树删除算法分析

1. 二叉排序树的结点删除定义

在二叉查找树上的删除一个结点，要求删除结点后，仍保持二叉排序树的结构特点不变。

2. 二叉排序树的结点删除方法

设被删结点为p,其双亲结点为f，则p可能有以下4种情况：

(1) p为叶结点

(2) p只有左子树

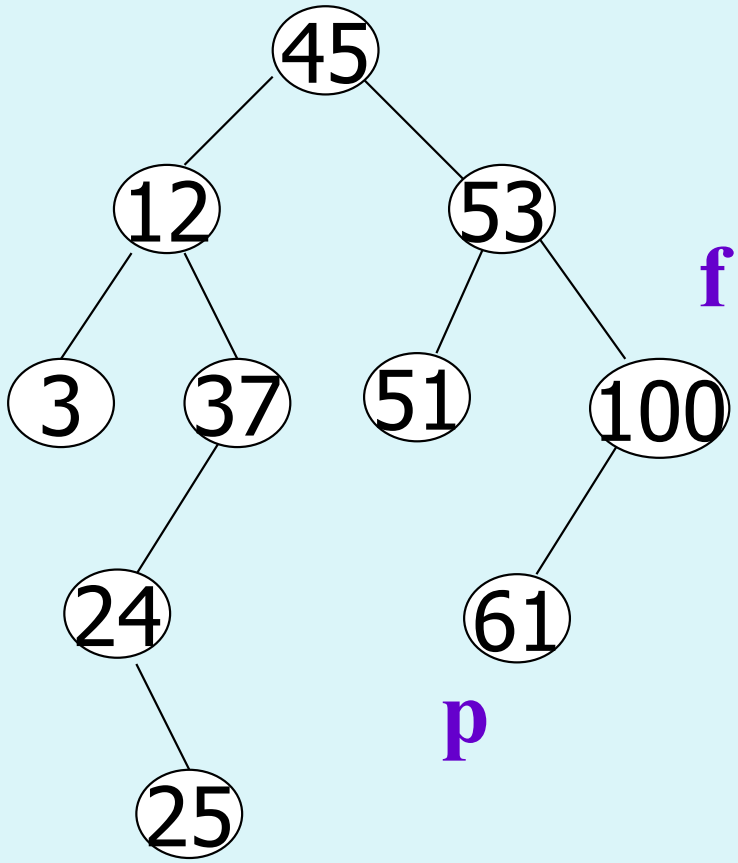
(3) p只有右子树

(4) p左右子树都有

二叉排序树的删除

2. 二叉排序树的结点删除方法

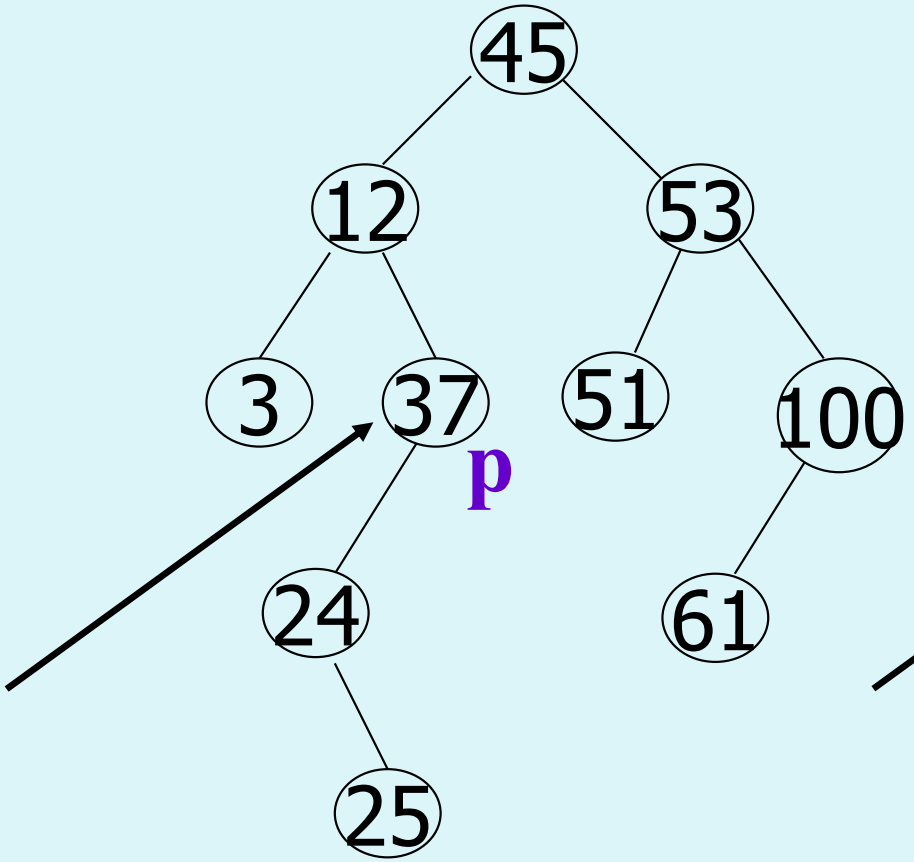
(1) p为叶结点,如右图所示:

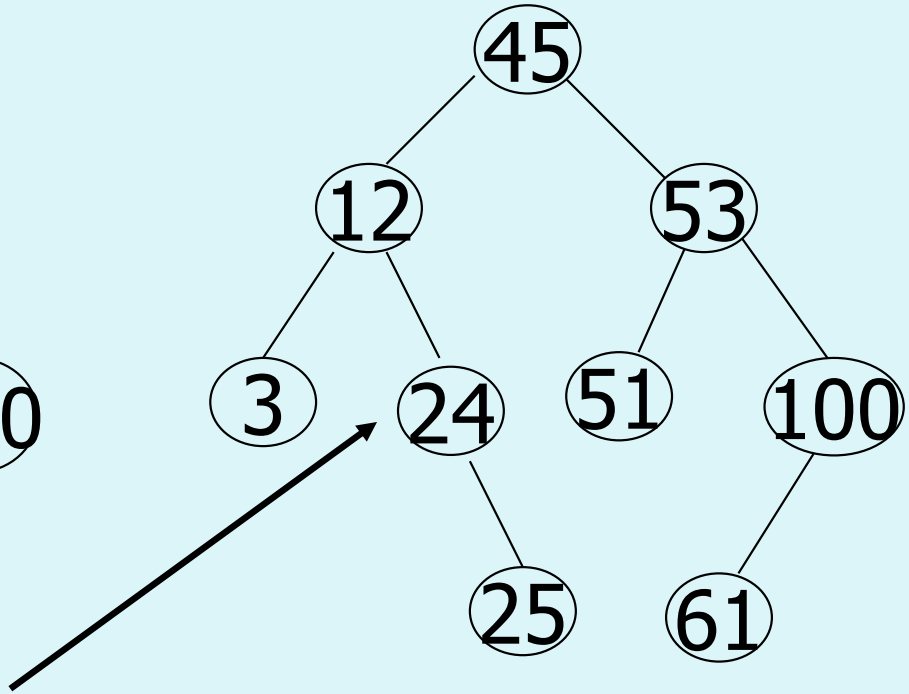


删除方法:

释放结点p，修改其父结点f的相应指针。

2. 二叉排序树的结点删除方法

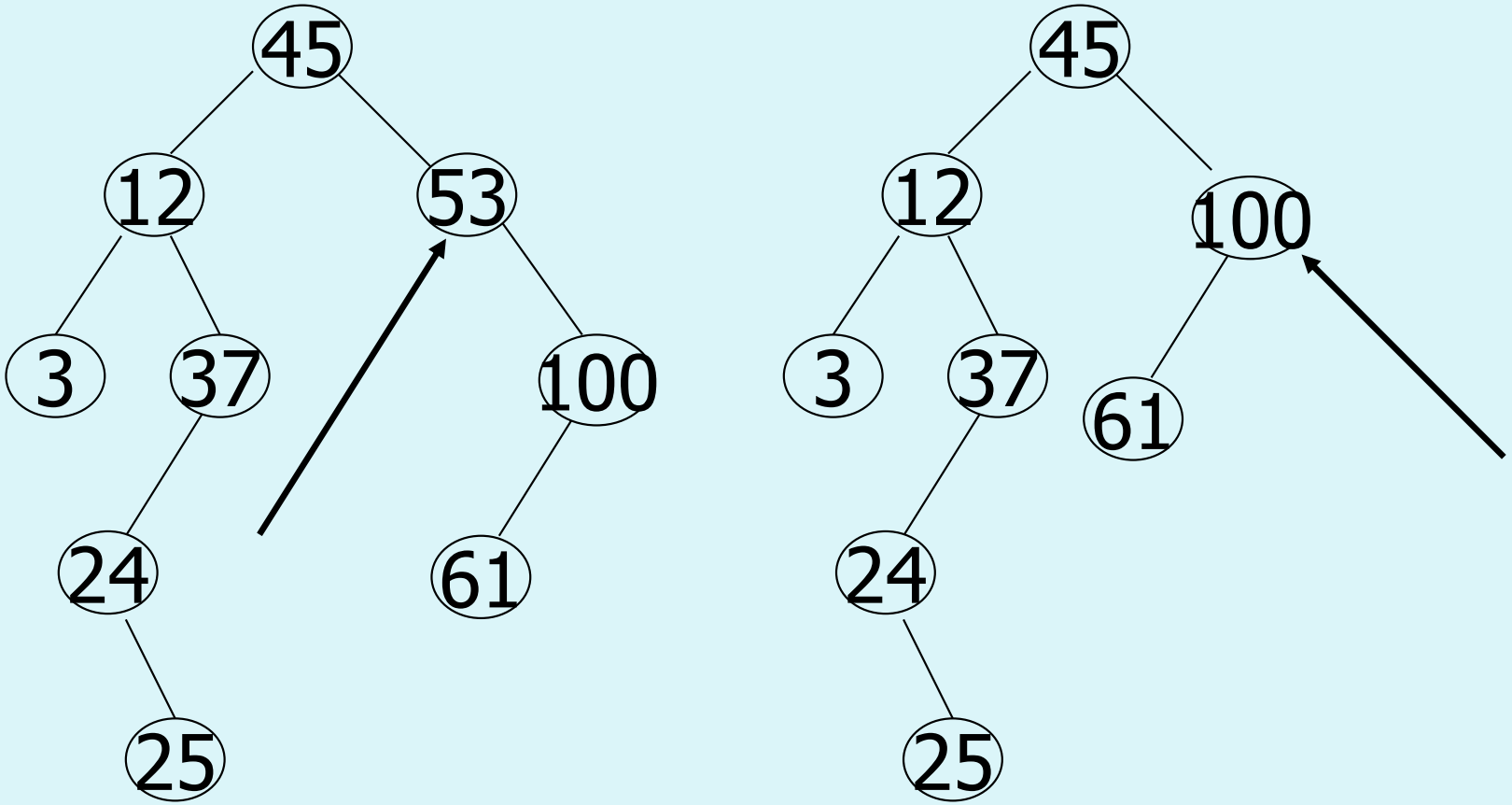




(2) p只有左子树，如上图 所示：

删除方法：释放结点p，p的左子树顶替p点的位置。

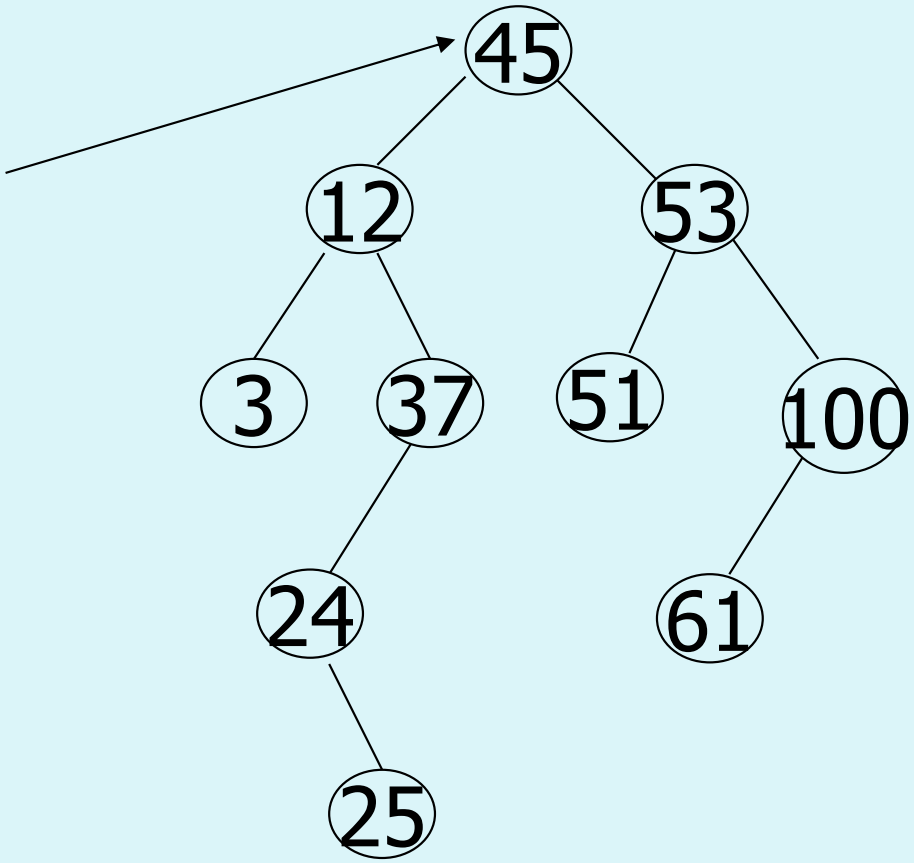
2. 二叉排序树的结点删除方法



(3) p只有右子树，如上图 所示：

删除方法：释放结点p，p的右子树顶替p点的位置。

2. 二叉排序树的结点删除方法



删除方法：

把左子树作为右子树中最小结点的左子树。

或者把右子树作为左子树中最大结点的右子树。

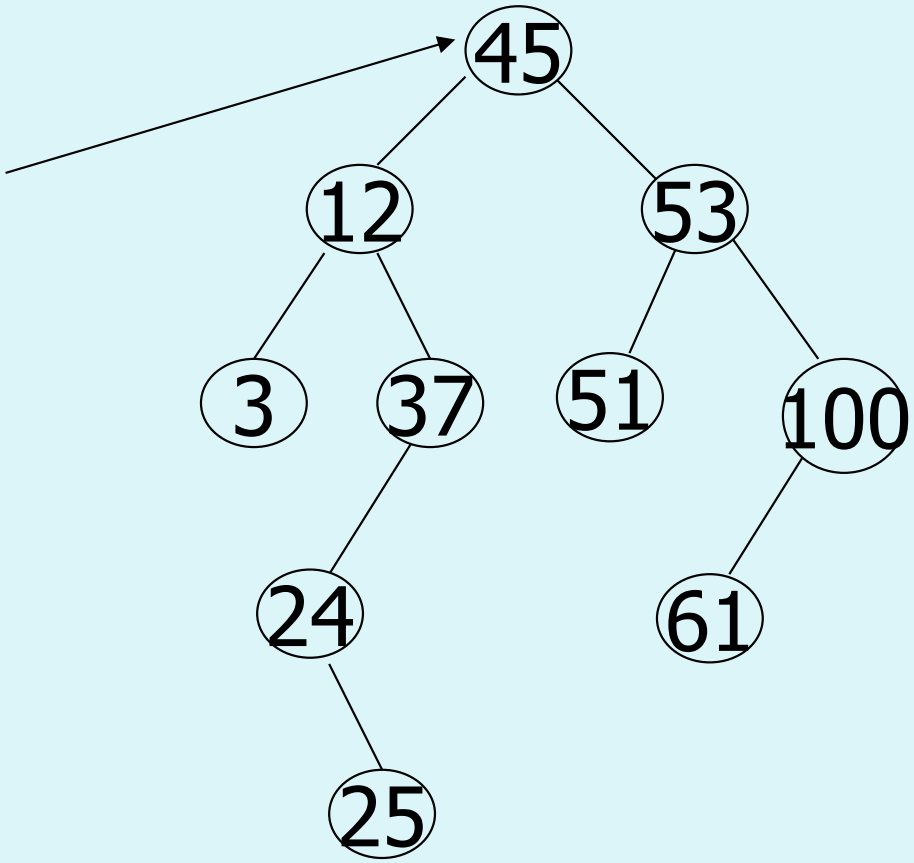
缺点：

增加了树的高度。

(4) p既有左子树，也有右子树，

如上图 所示：

2. 二叉排序树的结点删除方法



改进删除方法：

找一个结点顶替它的位置。

能够顶替它的结点是左子树中最大的，右子树中最小的。

我们用左子树最大的结点来顶替。

(4) p既有左子树，也有右子树，

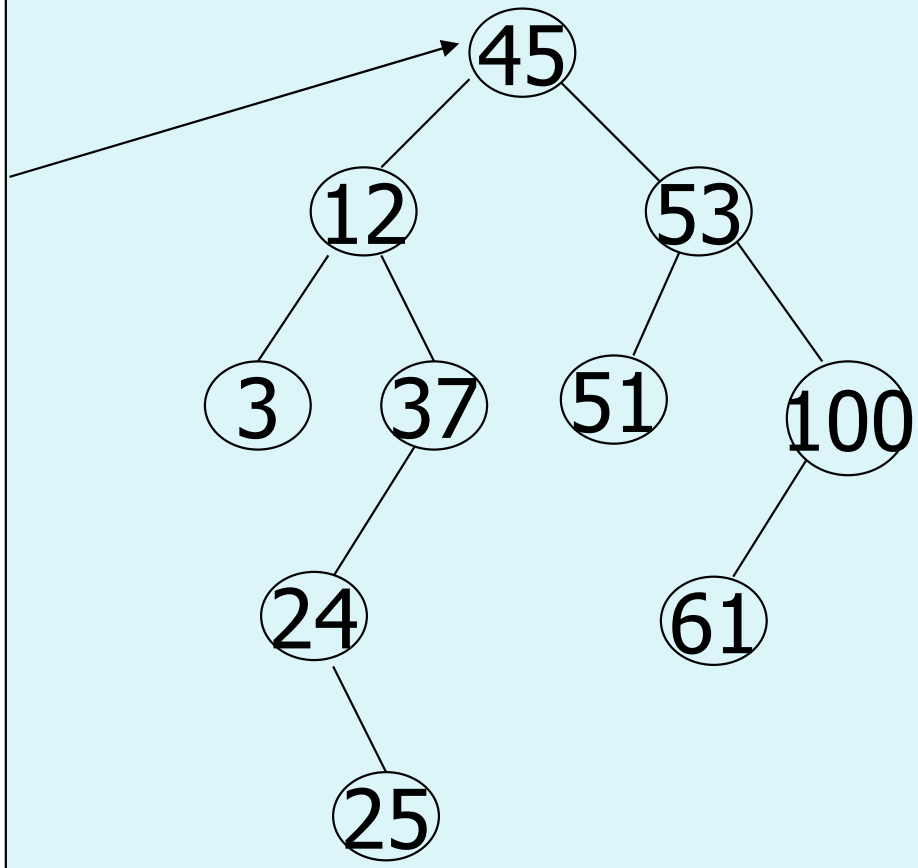
如上图 所示：

3. 二叉排序树的结点删除算法

```
Status DeleteBST(BiTree &T,  KeyType key)
{
    if (!T) return FALSE;
    else {
        if (EQ(key, T->data.key))
            return Delete(T);
        else if (LT(key, T->data.key))
                return  DeleteBST(T->lchild, key);
            else return DeleteBST(T->rchild, key);
  }
} // DeleteBST
```

3. 二叉排序树的结点删除算法

```
Status Delete(BiTree &p)
{ BiTree q, s;
  if (!p->rchild)
      {q = p;  p = p->lchild;  free(q);}
  else if (!p->lchild)
      {q = p;  p = p->rchild;  free(q);}
      else
     {q = p;  s = p->lchild;
      while (s->rchild)
          {q = s;  s = s->rchild; }
      p->data = s->data;
      if (q != p) q->rchild = s->lchild;
     else q->lchild = s->lchild;
      free(s);
      }
  return TRUE;
} // Delete
```



4. 二叉排序树的删除算法分析

(1)二叉排序树的删除算法的时间复杂性与查找算法的时间复杂性相同;

(2)最好情况是O（$\log_2 n$）;最坏情况是O(n);平均情况是O($\log_2 n$)。

单选题 1分

设置

用n 个关键字构造的一棵二叉排序树经过 i 次关键字比较成功找到的元素个数最多为 （ ）。

A i

B 2^{i}

C 2^{i-1}

D $2^{i}-1$

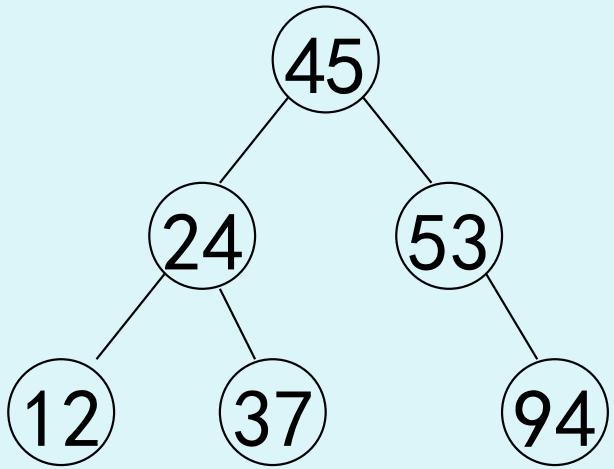
提交

8.2.1 二叉排序树和平衡二叉树

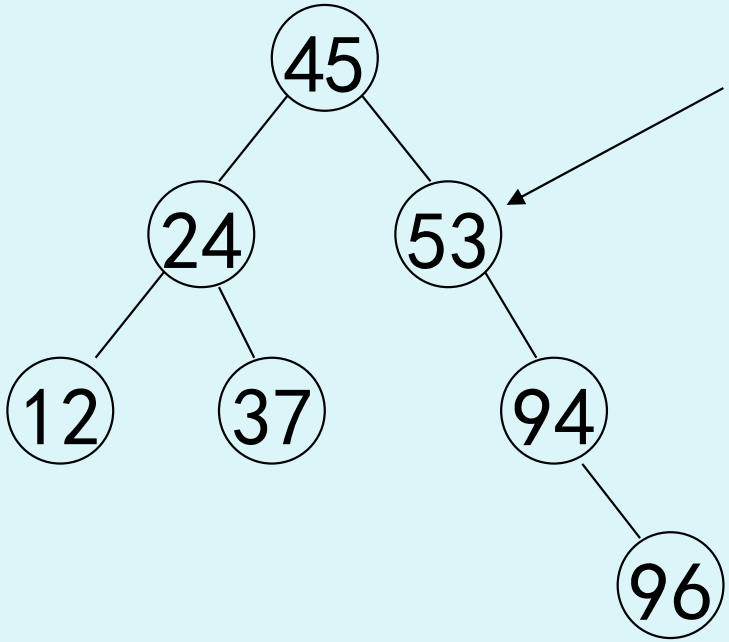
- 二叉排序树的定义
- 二叉排序树的结点查找
- 二叉排序树的结点插入
- 二叉排序树的结点删除
- 平衡二叉树的构建

1. 平衡二叉树的定义
2. 平衡因子的定义
3. 平衡二叉树构造思想
4. 平衡二叉树构造实例
5. 平衡二叉树构造算法

1. 平衡二叉树定义



平衡二叉树



非平衡二叉树

1. 平衡二叉树定义

平衡二叉树又称AVL树

一棵AVL树或者是空树，或者是具有下列性质的二叉排序树：它的所有节点的左子树和右子树的高度之差的绝对值不超过1。

2. 平衡因子的定义

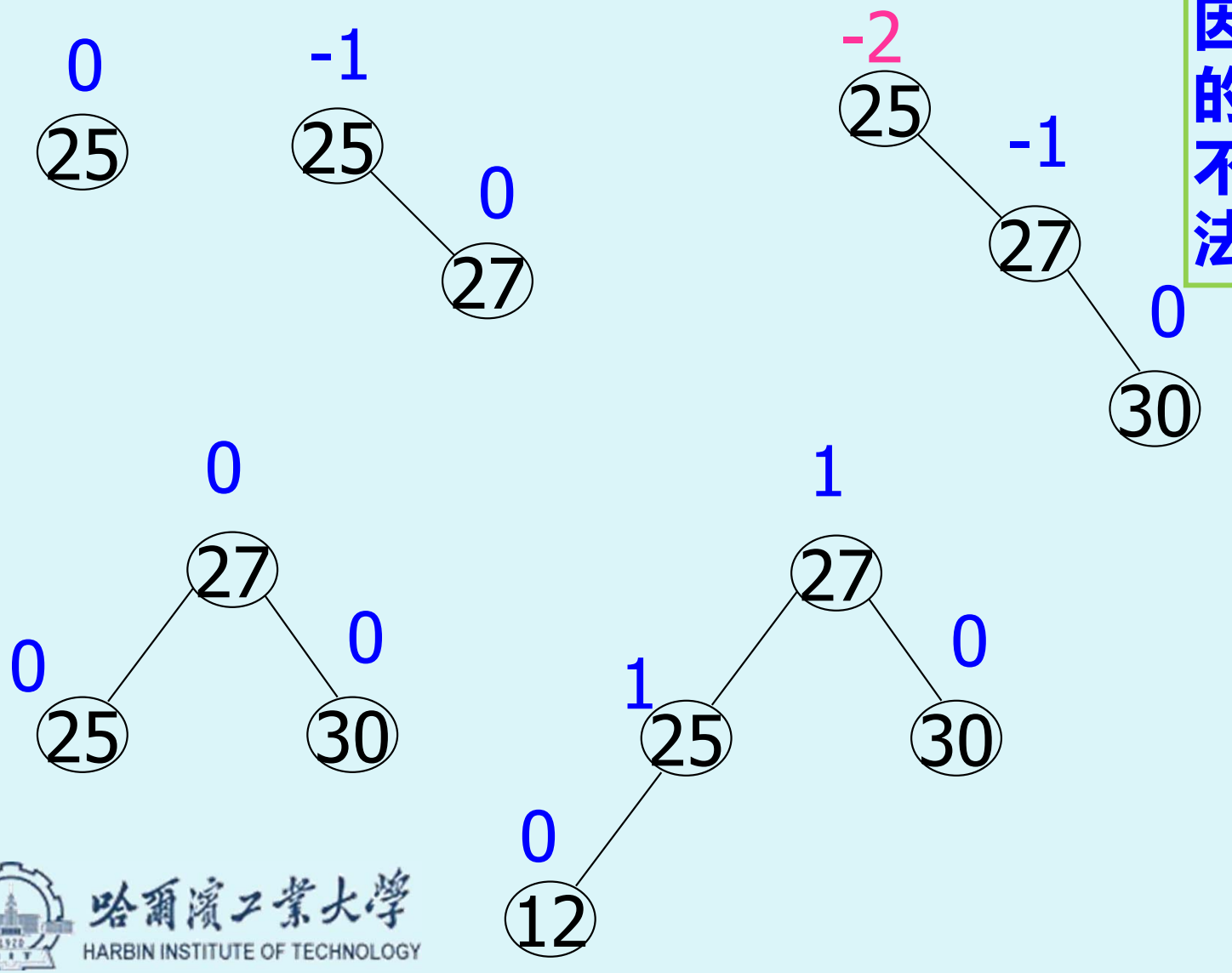
平衡因子(BF--Balance Factor)：任一结点的左子树的高度减去右子树的高度所得的高度差称为该结点的平衡因子BF。

根据AVL树的定义，任一结点的平衡因子只能取 -1，0 和 1。

一棵平衡二叉排序树如果有n个结点，其高度可保持在$O(\log_2 n)$，平均查找长度也可保持在$O(\log_2 n)$。

3.平衡二叉树的构造示例演示

例：给定序列25、27、30、12、11、18、14、20、15、22，构造其AVL树。



因为插入右子树的右子树造成的不平衡，调整办法称作RR旋转。

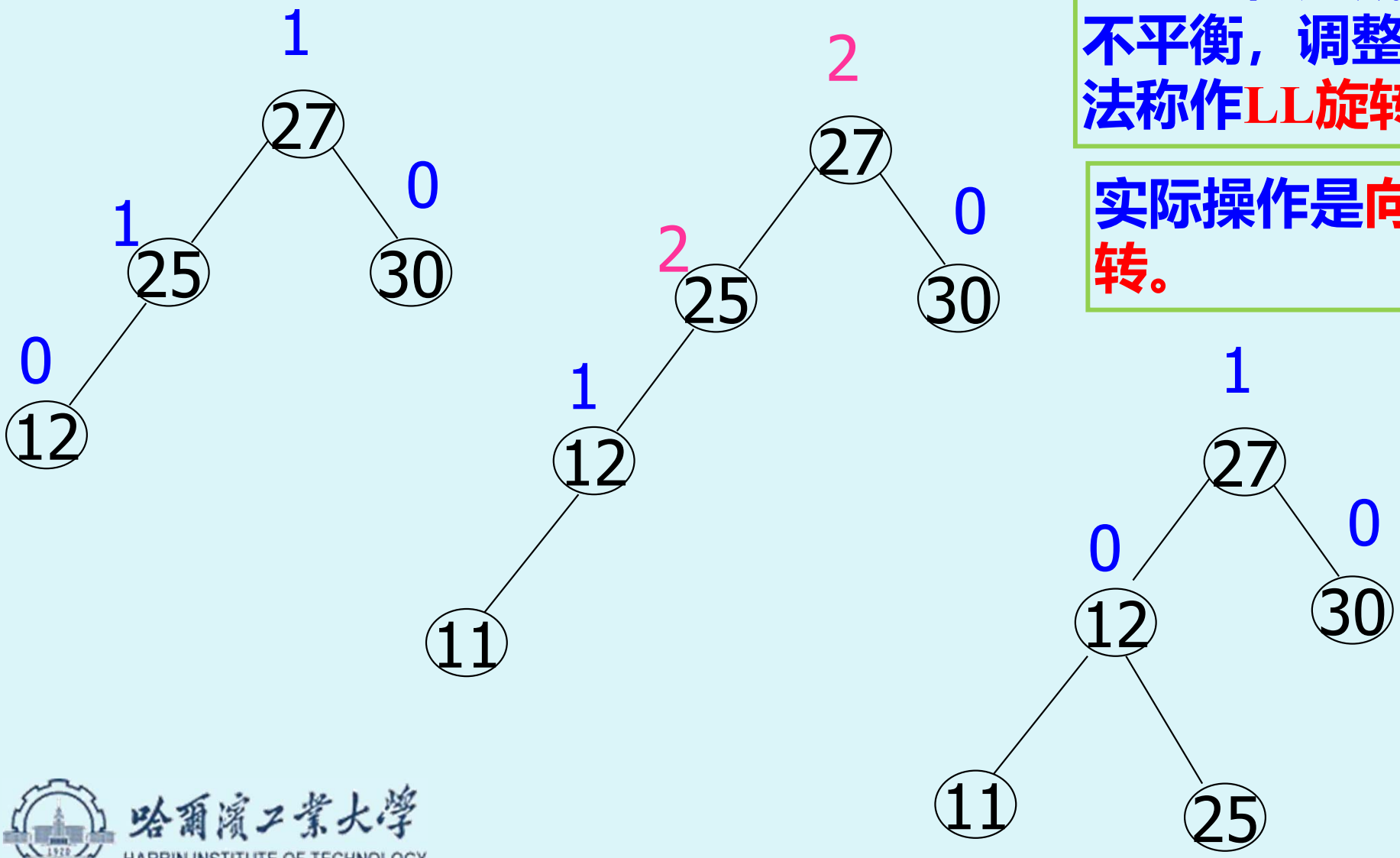
实际操作是向左旋转。

3.平衡二叉树的构造示例演示

例：给定序列25、27、30、12、11、18、14、20、15、22，求其AVL树

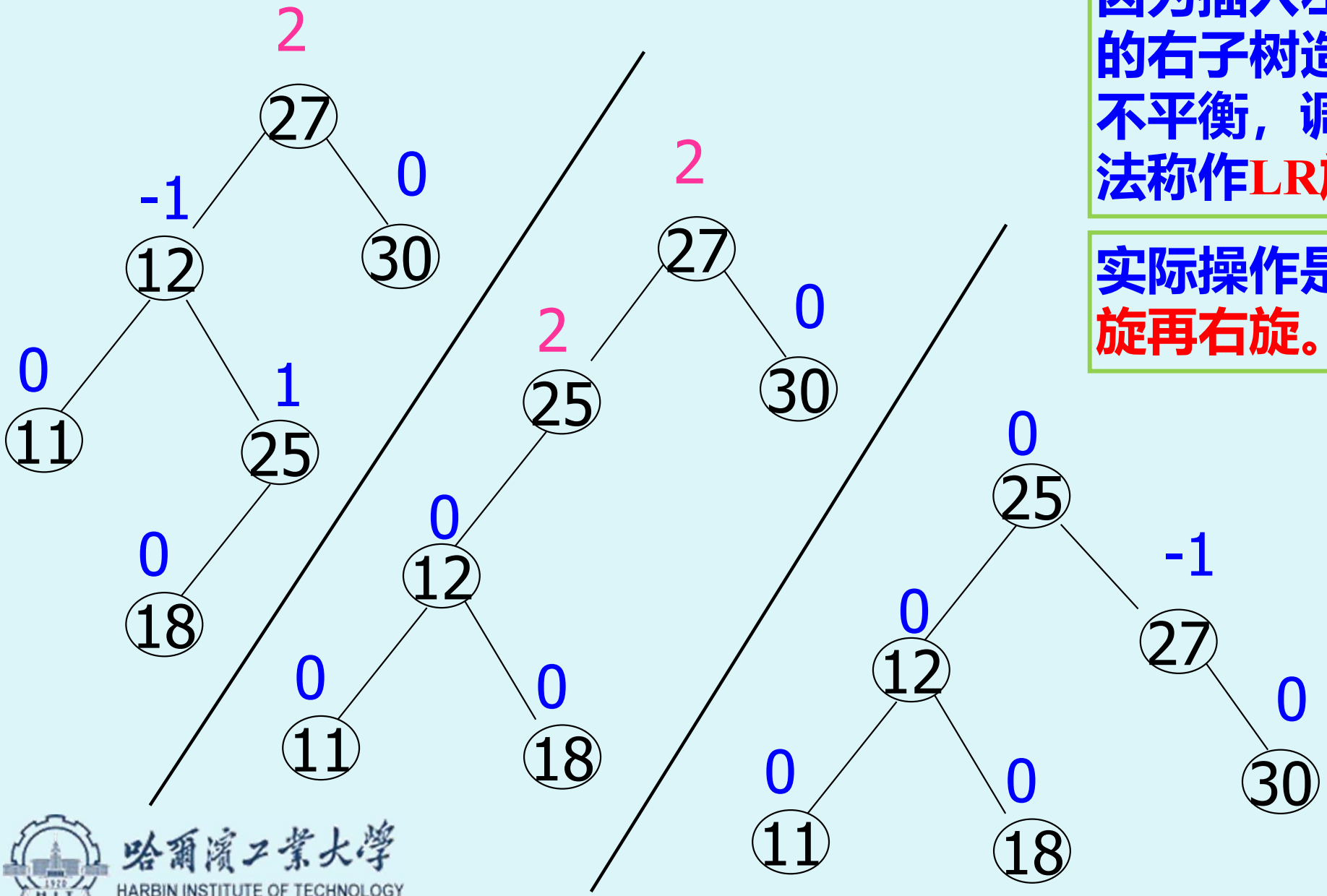
因为插入左子树的左子树造成的不平衡，调整办法称作LL旋转。

实际操作是向右旋转。



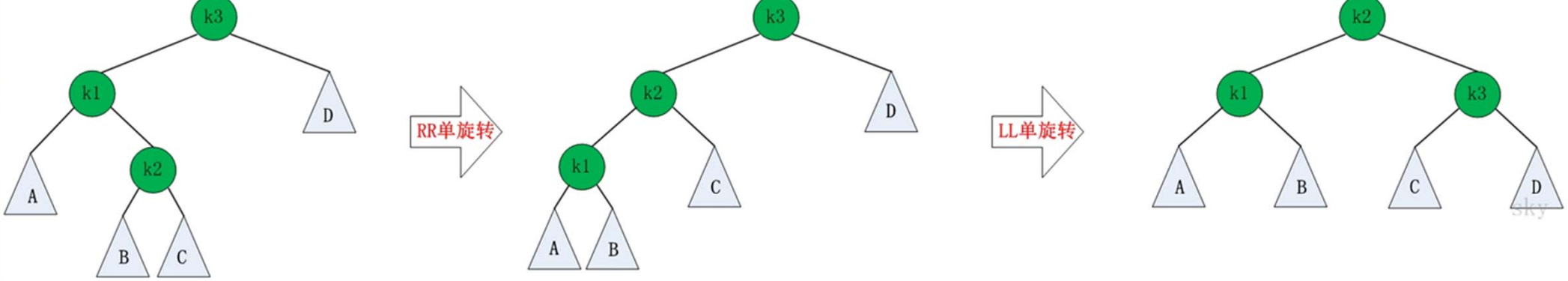
3.平衡二叉树的构造示例演示

例：给定序列25、27、30、12、11、18、14、20、15、22，求其AVL树



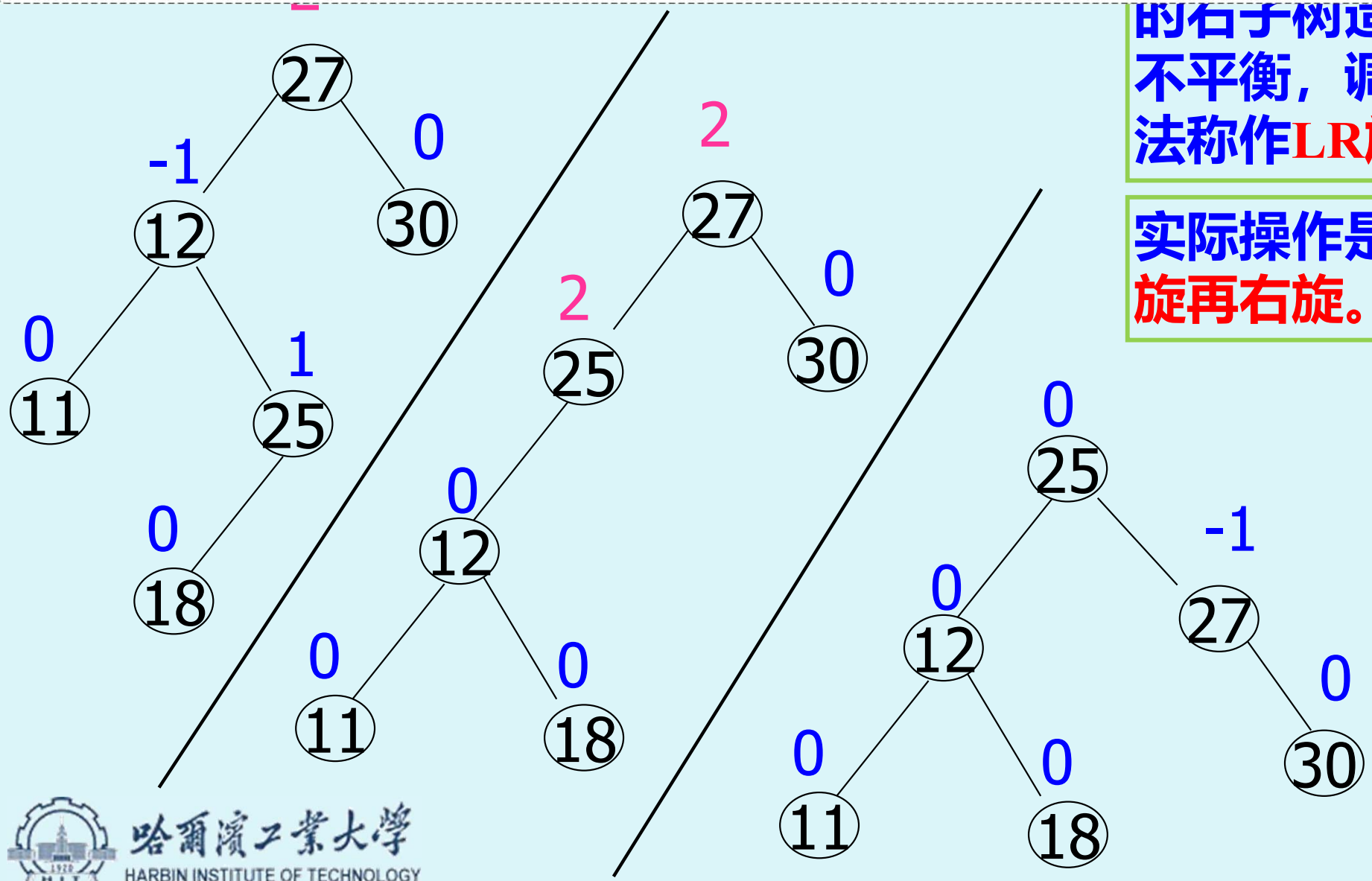
因为插入左子树的右子树造成的不平衡，调整办法称作LR旋转。

实际操作是先左旋再右旋。



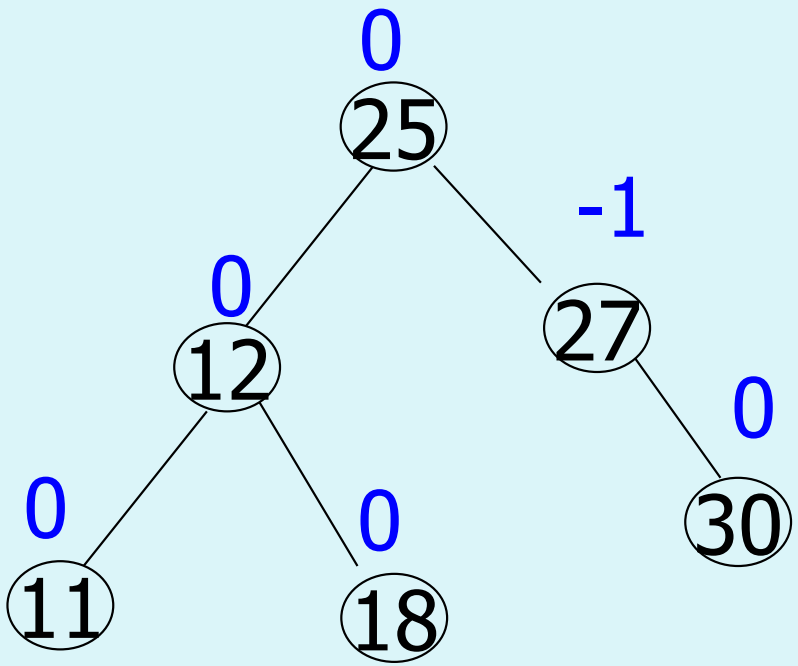
的右子树造成的不平衡，调整办法称作LR旋转。

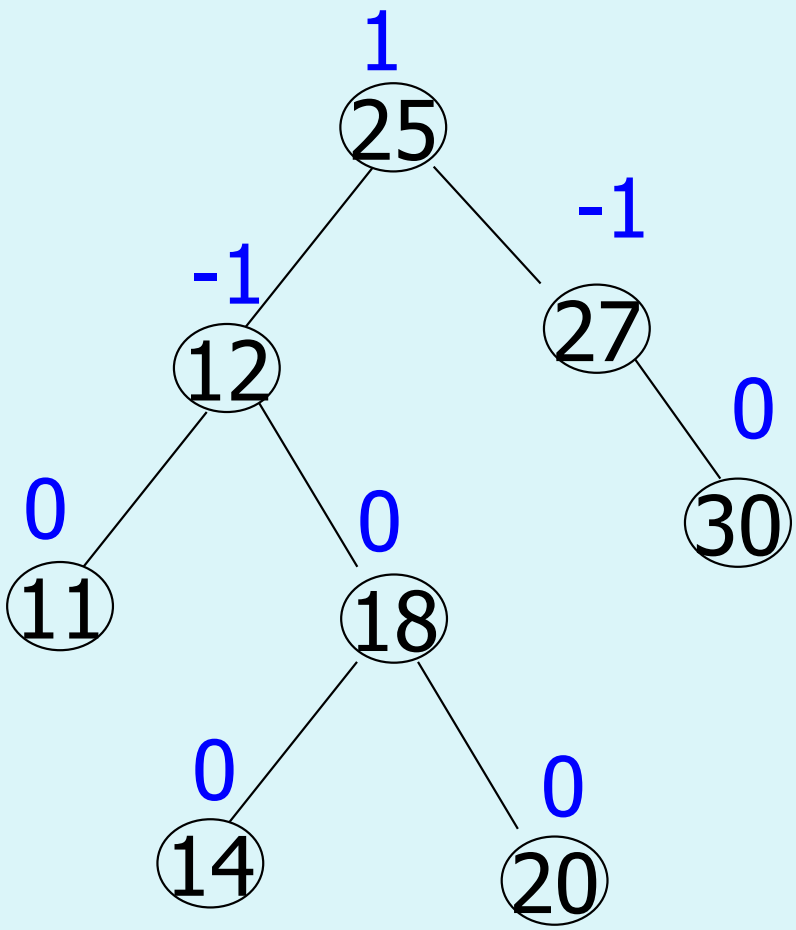
实际操作是先左旋再右旋。



3.平衡二叉树的构造示例演示

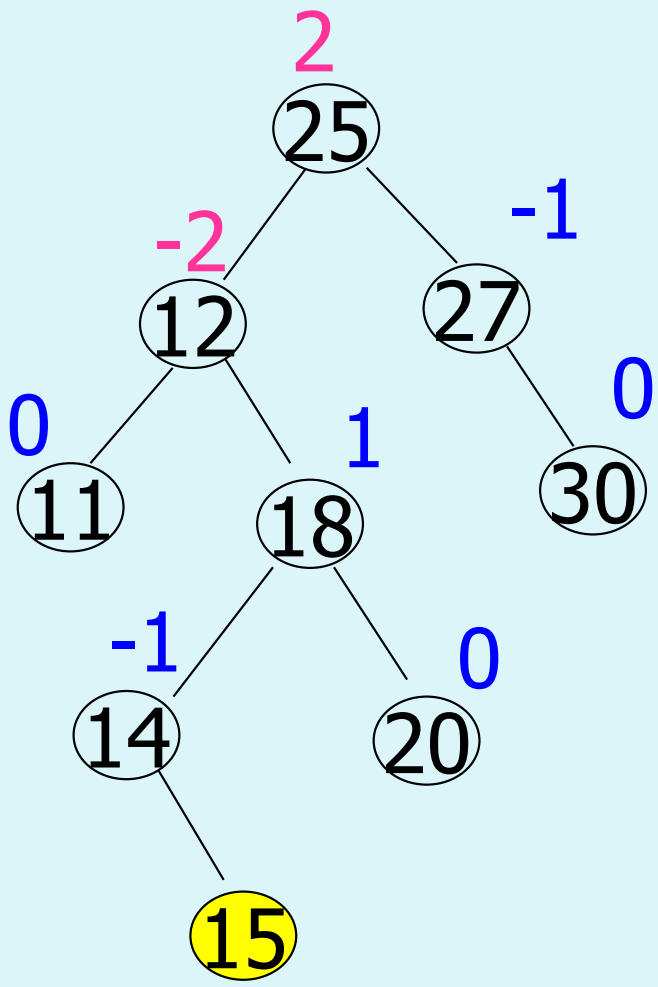
例：给定序列25、27、30、12、11、18、14、20、15、22，求其AVL树

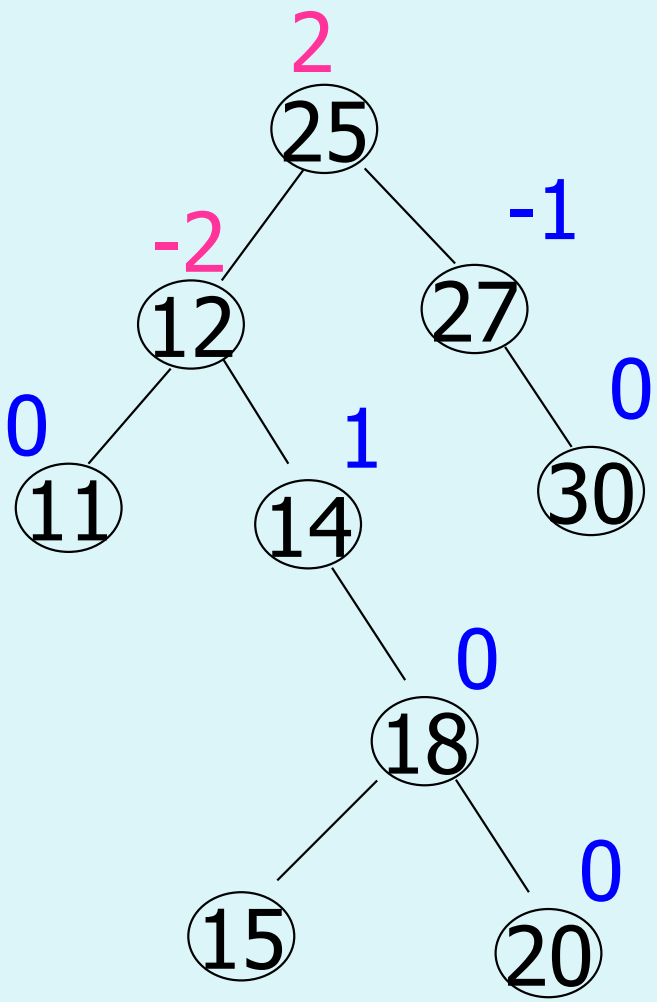




3.平衡二叉树的构造示例演示

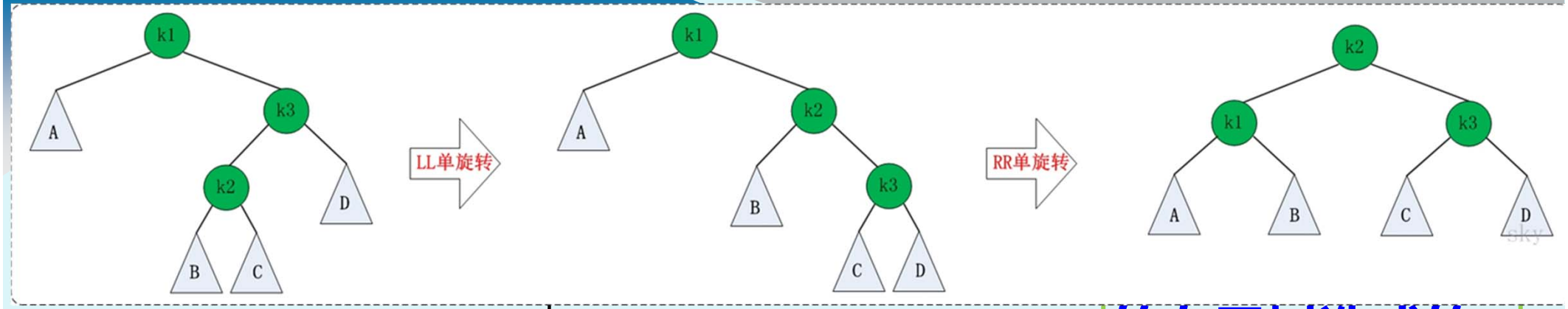
例：给定序列25、27、30、12、11、18、14、20、15、22，求其AVL树

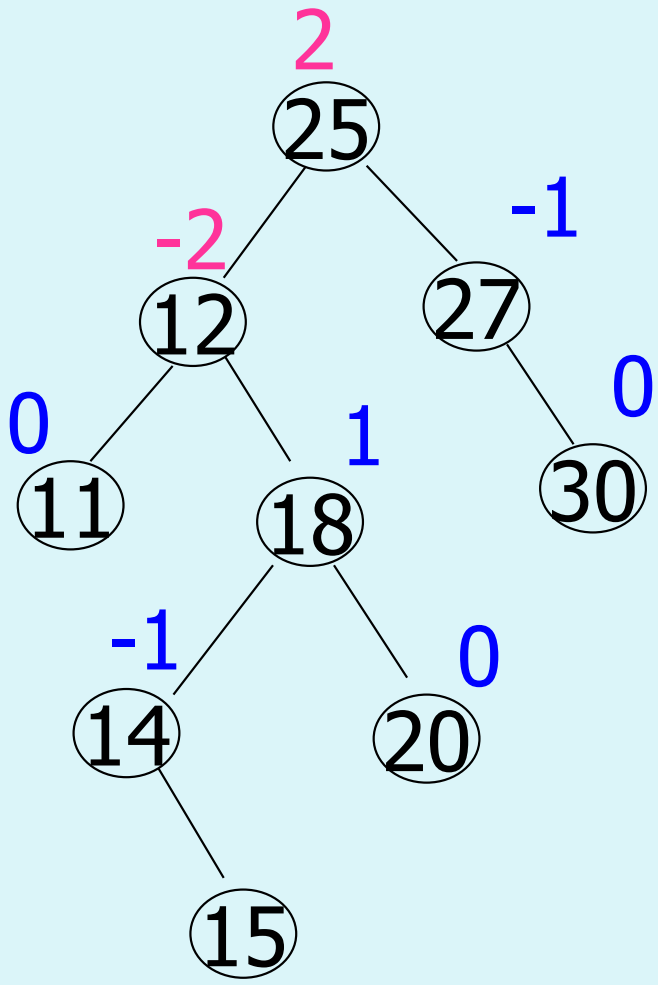


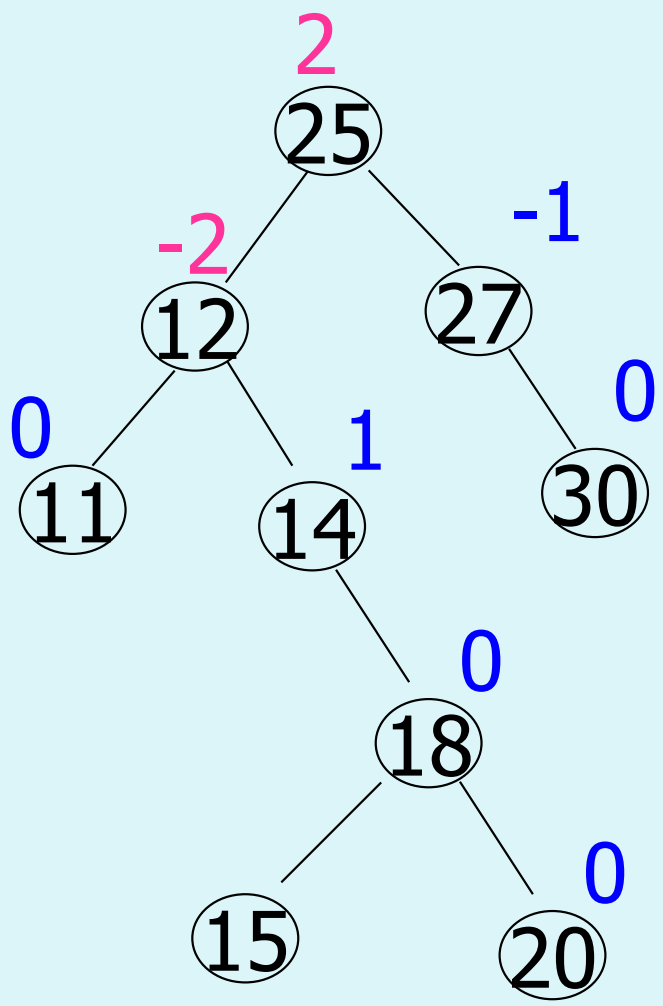


因为插入右子树的左子树造成的不平衡，调整办法称作RL旋转。

实际操作是先右旋再左旋。

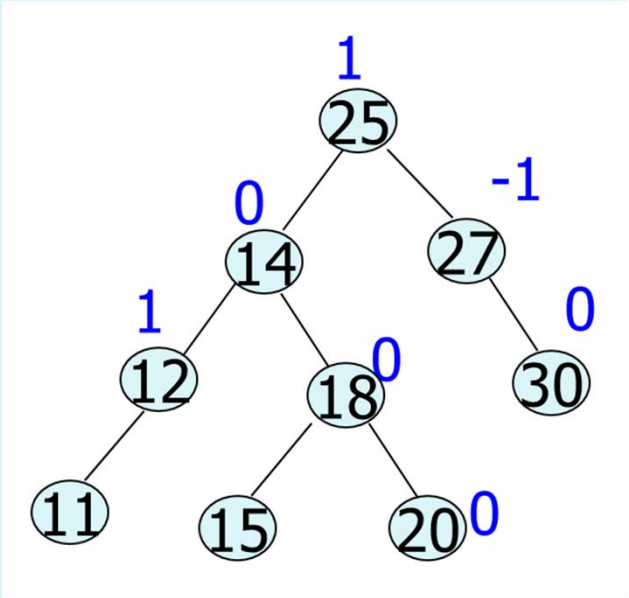






的左子树造成的不平衡，调整办法称作RL旋转。

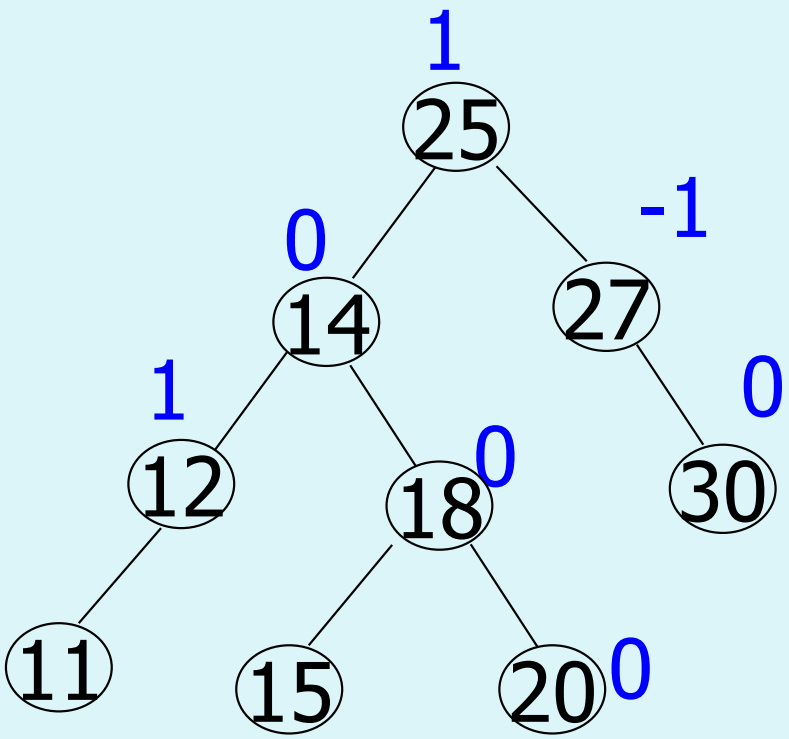
实际操作是先右旋再左旋。

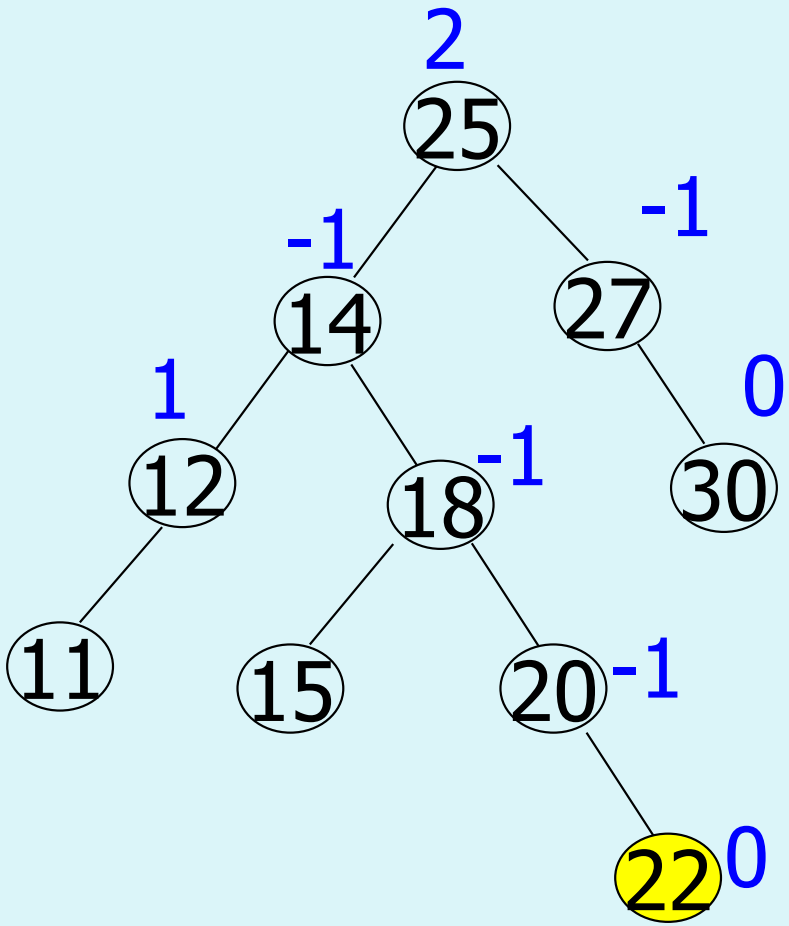


哈爾濱工業大學
HARBIN INSTITUTE OF TECHNOLOGY

3.平衡二叉树的构造示例演示

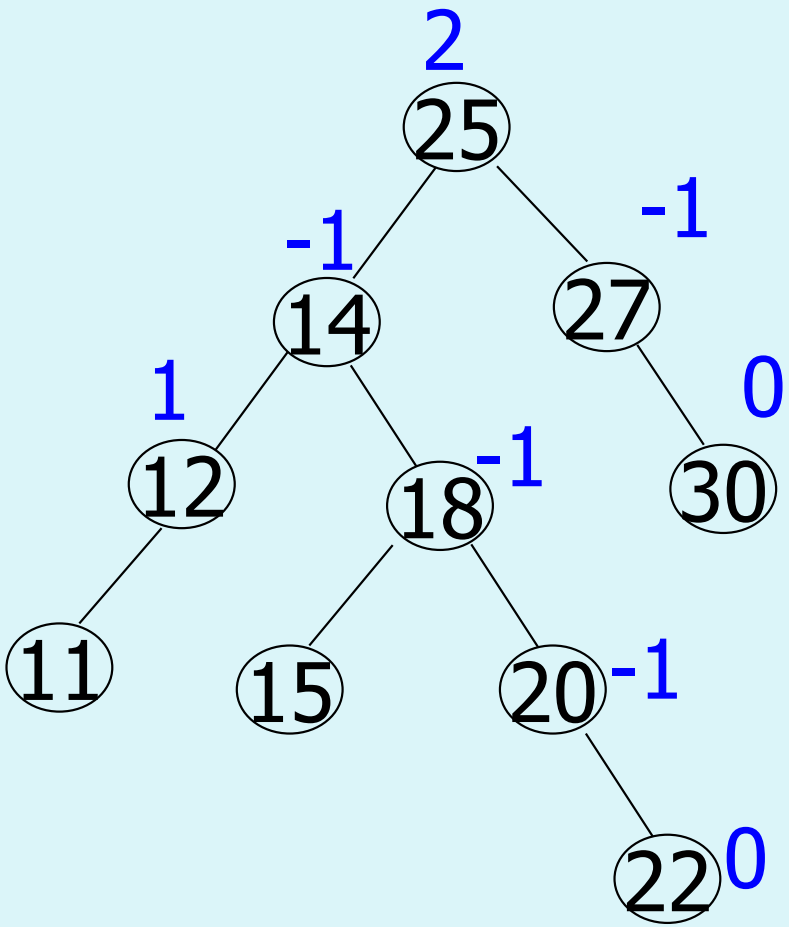
例：给定序列25、27、30、12、11、18、14、20、15、22，求其AVL树

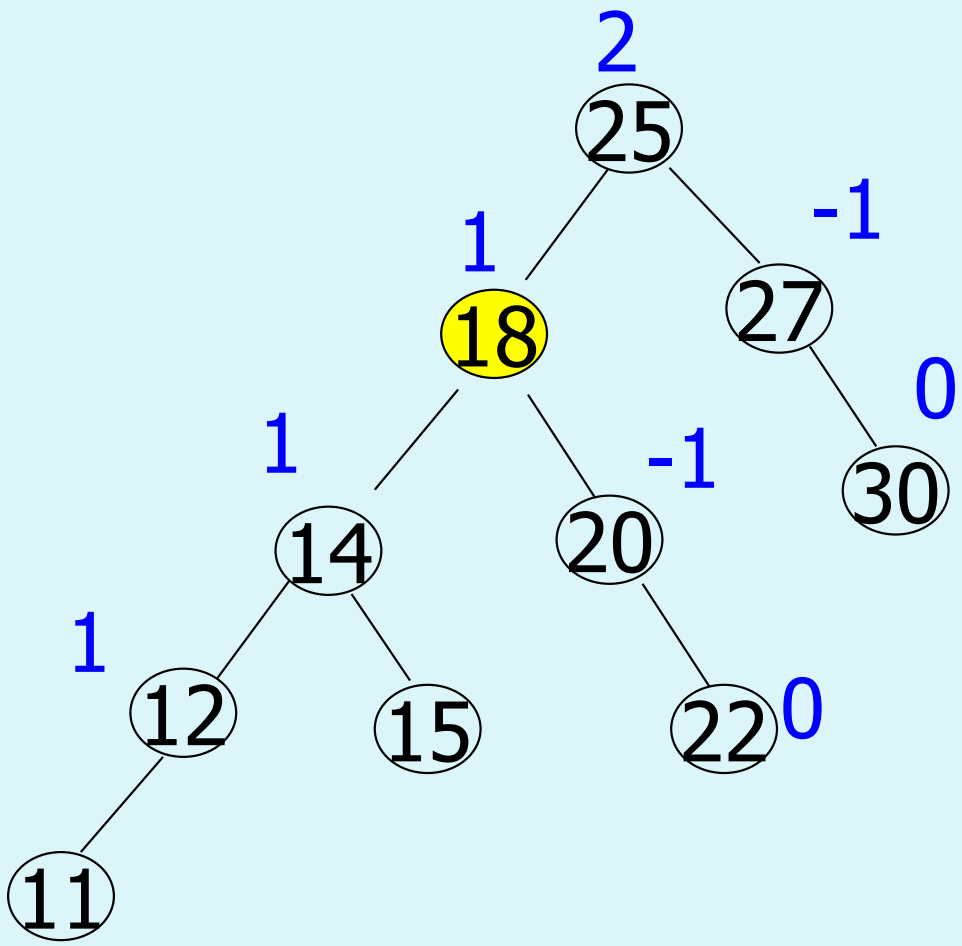




3.平衡二叉树的构造示例演示

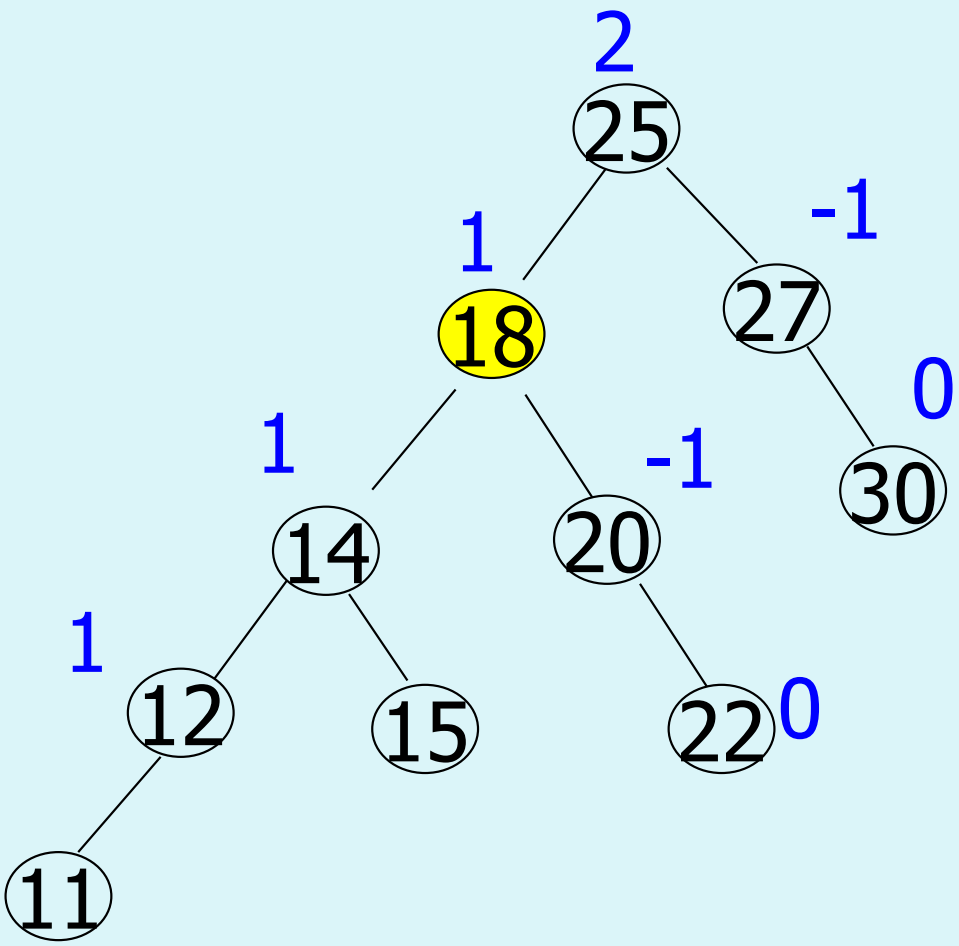
例：给定序列25、27、30、12、11、18、14、20、15、22，求其AVL树

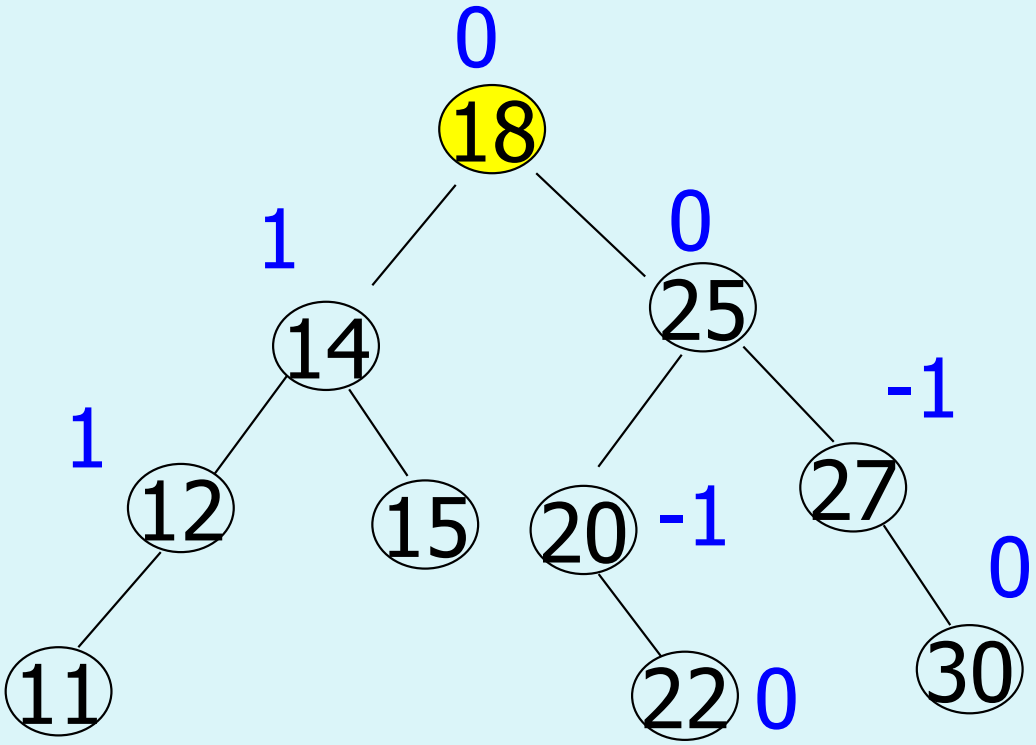


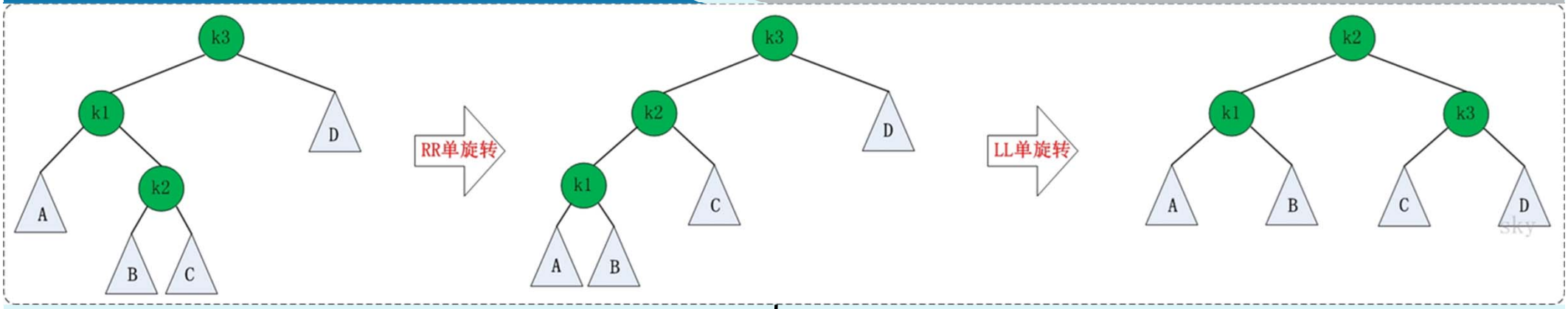


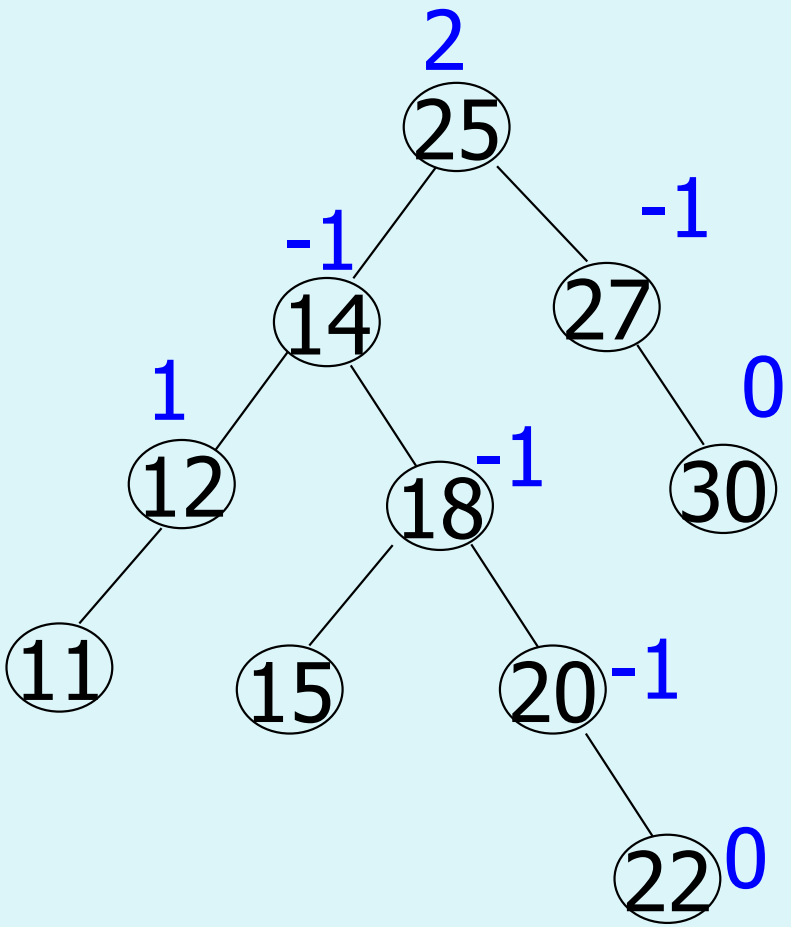
3.平衡二叉树的构造示例演示

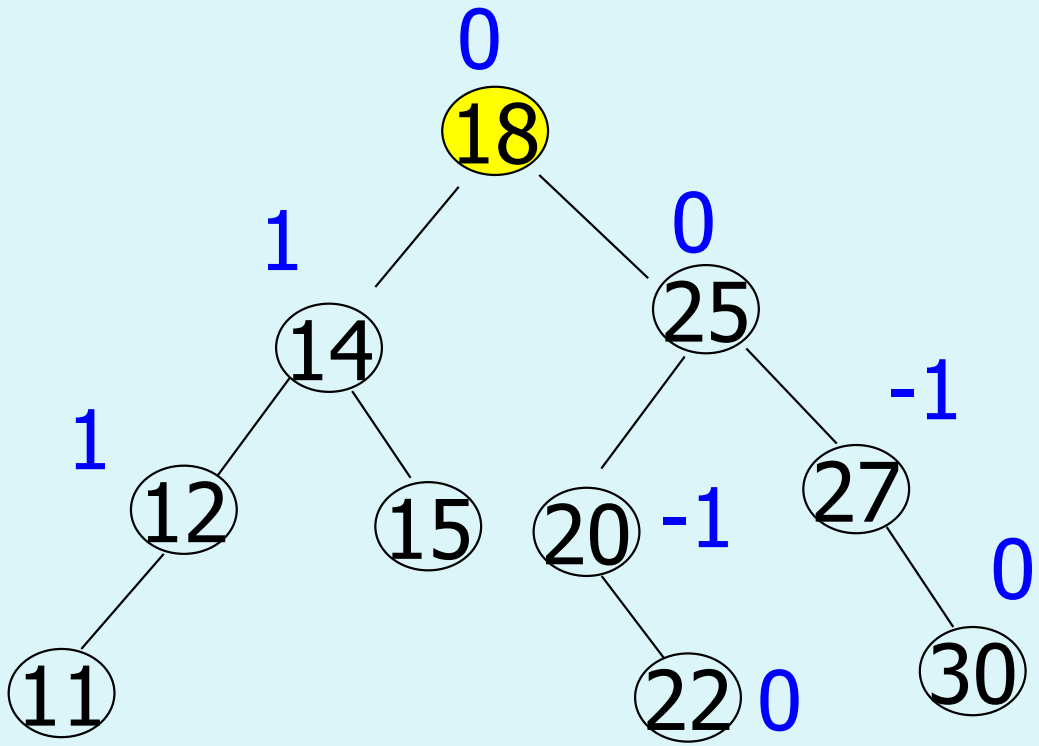
例：给定序列25、27、30、12、11、18、14、20、15、22，求其AVL树











哈爾濱工業大學
HARBIN INSTITUTE OF TECHNOLOGY

4.平衡二叉树的构造思想

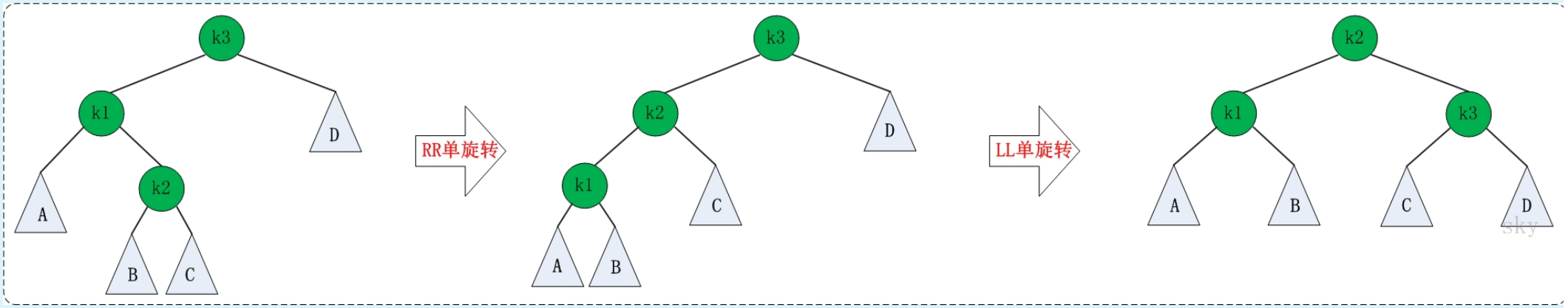
在构造过程中，每当插入一个新结点时，首先检查是否因插入而破坏了树的平衡性。

若是，则找出其中最小不平衡子树，在保持二叉排序树特性的前提下，调整最小不平衡子树中各结点之间的连接关系,以达到新的平衡。

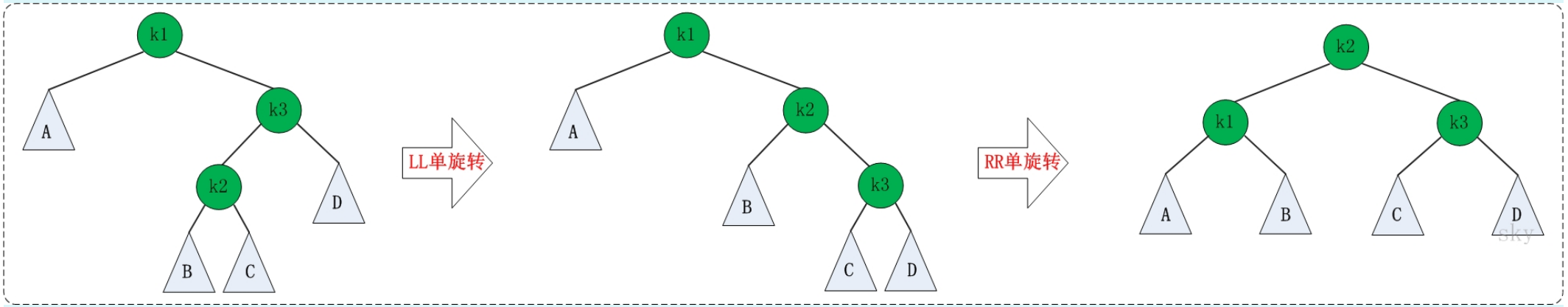
最小不平衡子树：以离插入结点最近，且平衡因子绝对值大于 1 的结点作为根的子树。

假设二叉排序树的最小不平衡子树的根结点为 A ，则有四种形态需要调整。

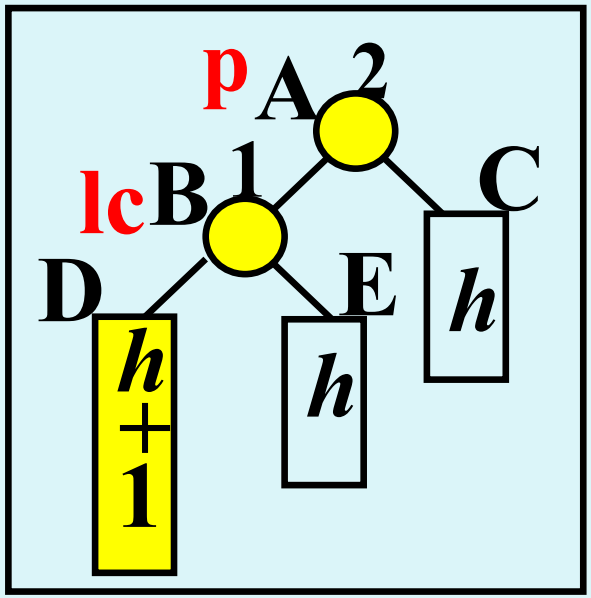
LR的旋转

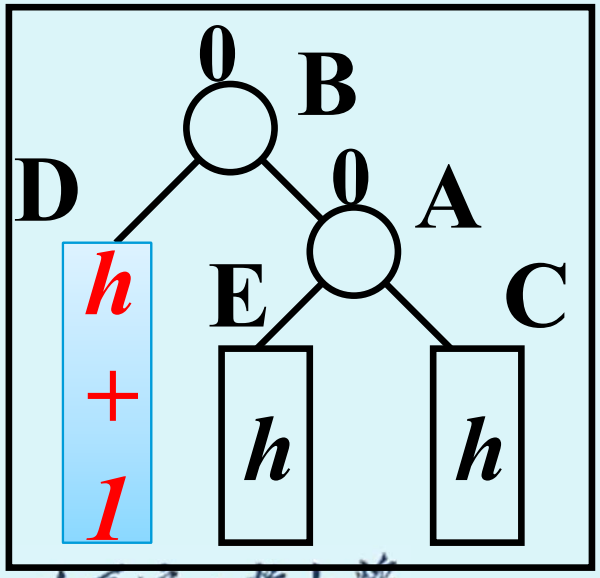

k3
k1
D
k2
A
B
C
RR单旋转
k3
k2
D
k1
C
A
B
LL单旋转
k2
k1
k3
A
B
C
D

RL的旋转


k1
A
k3
k2
D
B
C
LL单旋转
k1
A
k2
B
k3
C
D
RR单旋转
k2
k1
k3
A
B
C
D

平衡二叉树的构造算法

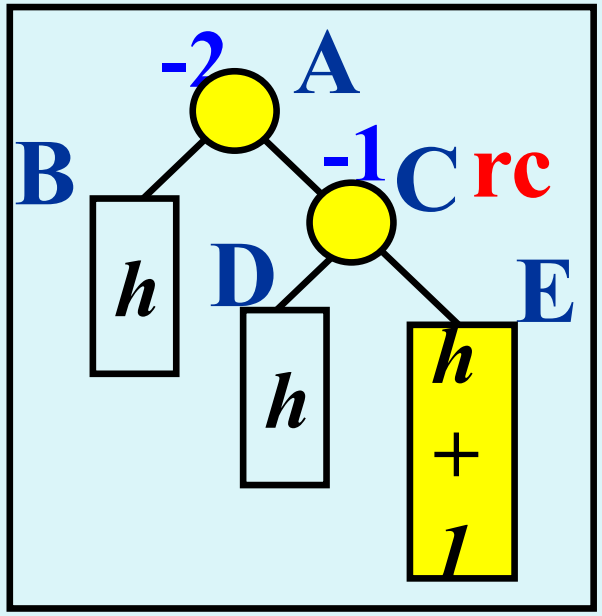


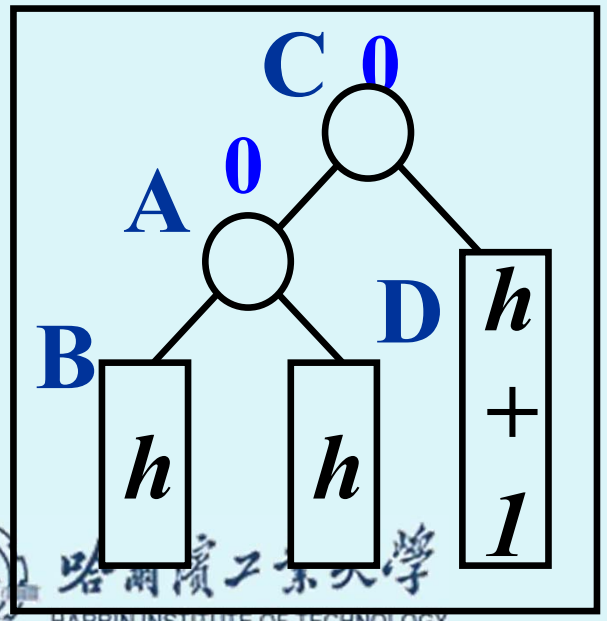


① LL型—右旋

```
void R_Rotate(BSTree &p) {
  BSTree lc;
  lc = p->lchild;
  p->lchild=lc->rchild;
  lc->rchild=p;
  p = lc;
  }
```

平衡二叉树的构造算法

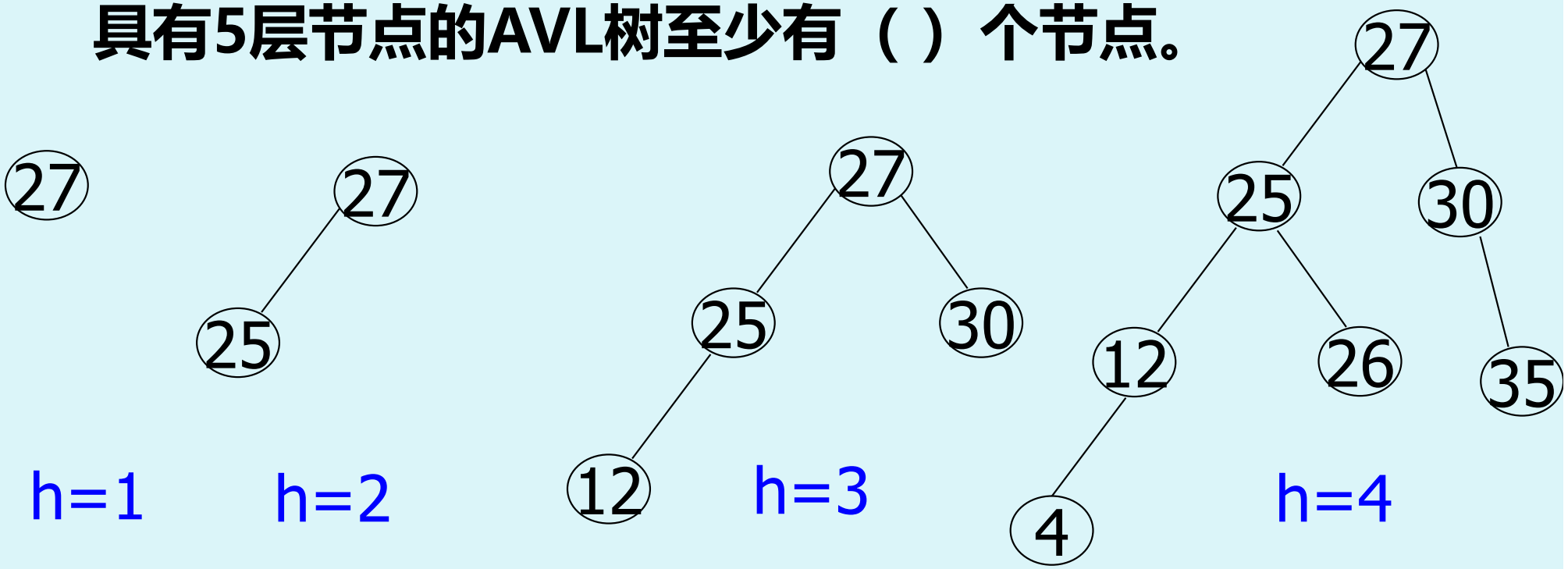




② RR型— 左旋

```
void L_Rotate(BSTree &p)
 {BSTree rc;
  rc = p->rchild;
  p->rchild=rc->lchild;
  rc->lchild = p;
  p = rc;
  }
```

具有5层节点的AVL树至少有（ ）个节点。



h=1 h=2 h=3 h=4

n=1 n=2 n=4 n=7

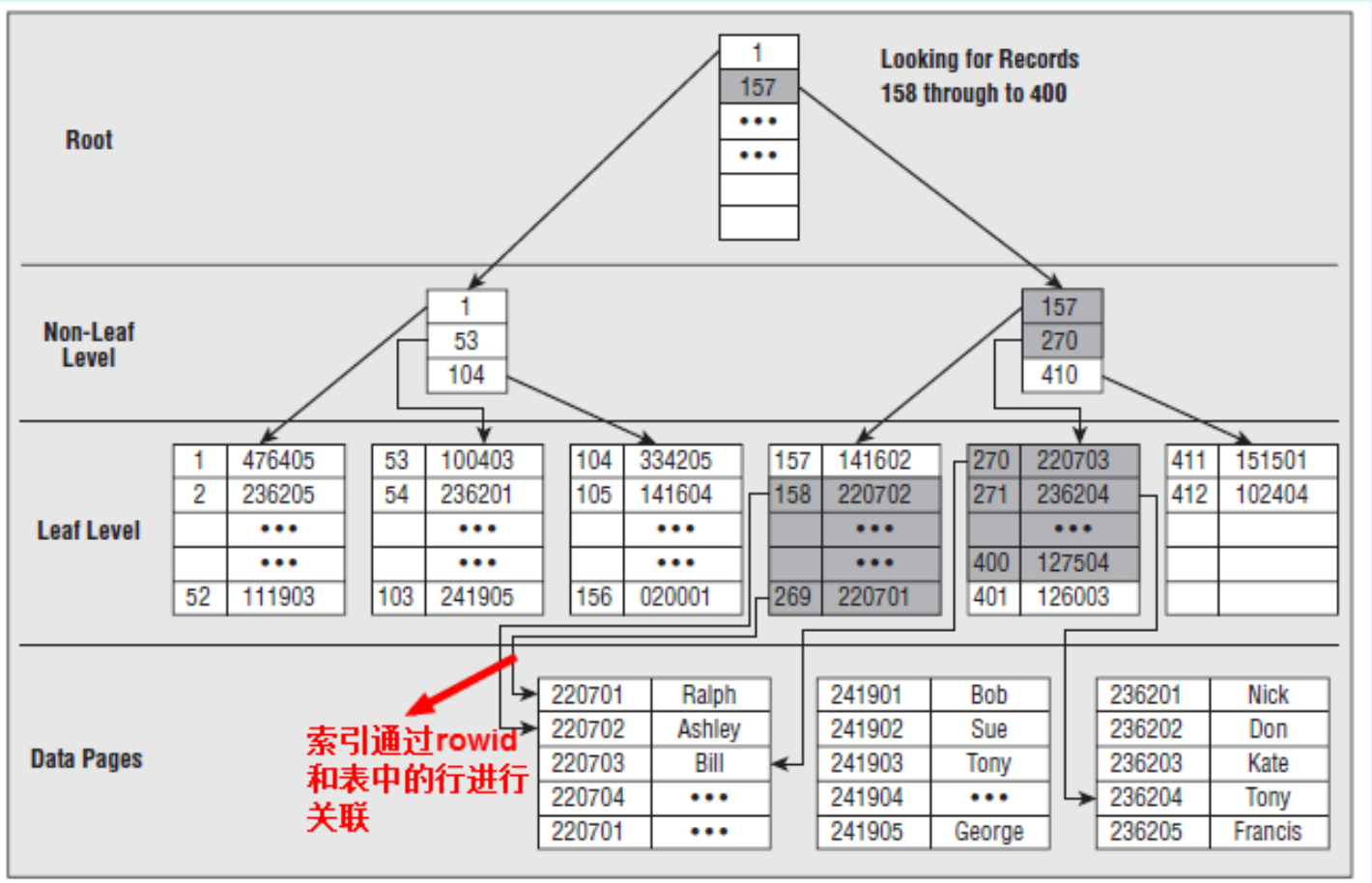
$N_h=N_{h-1}+N_{h-2}+1$

$N_5=12$

练习：

- 度为12的表(Jan, Feb, Mar, Apr, May, June, July, Aug, Sep, Oct, Nov, Dec)
- (1)试按表中元素的顺序依次插入一棵初始为空的二叉排序树，画出插入完成之后的二叉排序树，并求其在等概率的情况下查找成功的平均查找长度。
- (2)若对表中元素先进行排序构成有序表，求在等概率的情况下对此有序表进行折半查找时查找成功的平均查找长度。
- (3)按表中元素顺序构造一棵平衡二叉排序树，并求其在等概率的情况下查找成功的平均查找长度。

查找应用


Root
Looking for Records
158 through to 400
Non-Leaf Level
Leaf Level
Data Pages
索引通过rowid和表中的行进行关联

第8章 查找

预备知识

8.1 静态查找表

8.2 动态查找表

8.3 哈希表

8.3.1 什么是哈希表

8.3.2 哈希函数的构造方法

8.3.3 处理冲突的方法

8.3.4 哈希表的查找及其分析

第8章 查找

8.3.1 什么是哈希表

8.3.2 哈希函数的构造方法

8.3.3 处理冲突的方法

8.3.4 哈希表的查找及其分析

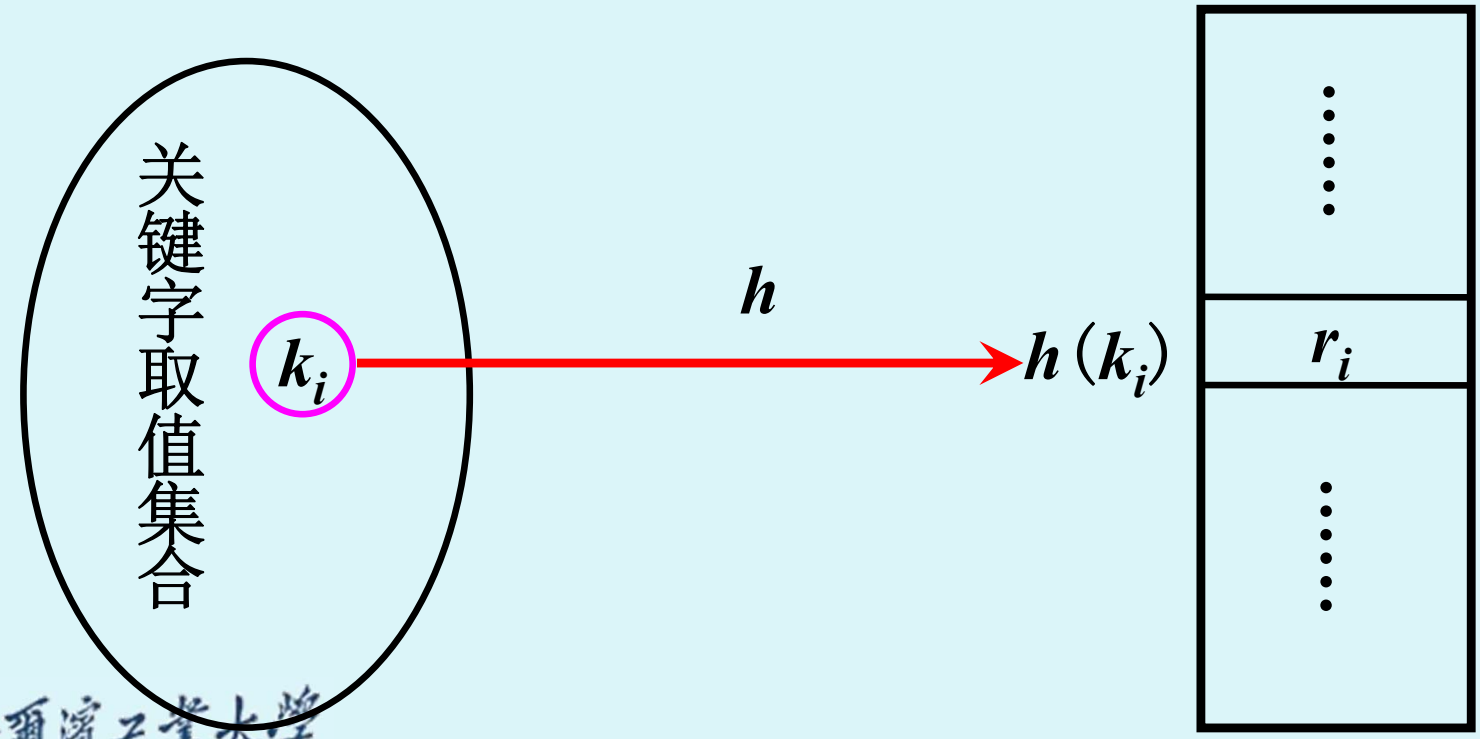
1. 哈希表的引入
2. 哈希表的思想
3. 哈希表的定义
4. 哈希表的有关概念

哈希（散列）技术

- 查找操作要完成什么任务？
  - 对于待查值k，通过比较，确定k在存储结构中的位置
- 基于关键字比较的查找的时间性能如何？
  - 其时间性能为$O(\log n)$~$O(n)$。
  - 用判定树可以证明，基于关键字比较的查找的平均和最坏情况下的比较次数的下界是$\log n+O(1)$，即$\Omega(\log n)$
  - 要向突破此下界，就不能仅依赖于基于比较来进行查找。
- 能否不用比较，通过关键字的取值直接确定存储位置？
  - 在关键字值和存储位置之间建立一个确定的对应关系

哈希（散列）技术

- 散列技术的基本思想
  - 把记录（元素）的存储位置和该记录的关键字的值之间建立一种映射关系。关键字的值在这种映射关系下的像，就是相应记录在表中的存储位置。
  - 散列技术在理想情况下，无需任何比较就可以找到待查的关键字，其查找的期望时间为O(1)。

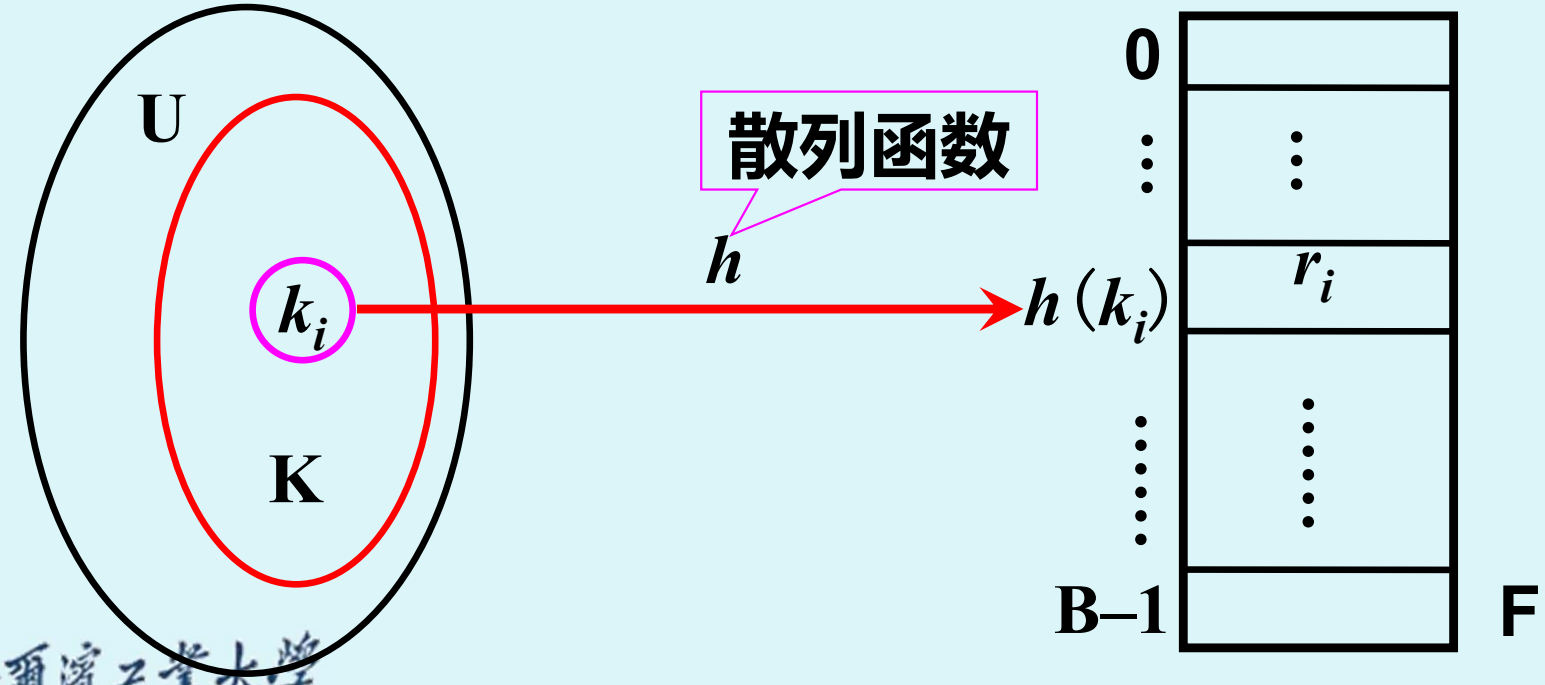


哈希（散列）技术

- 散列技术的相关概念

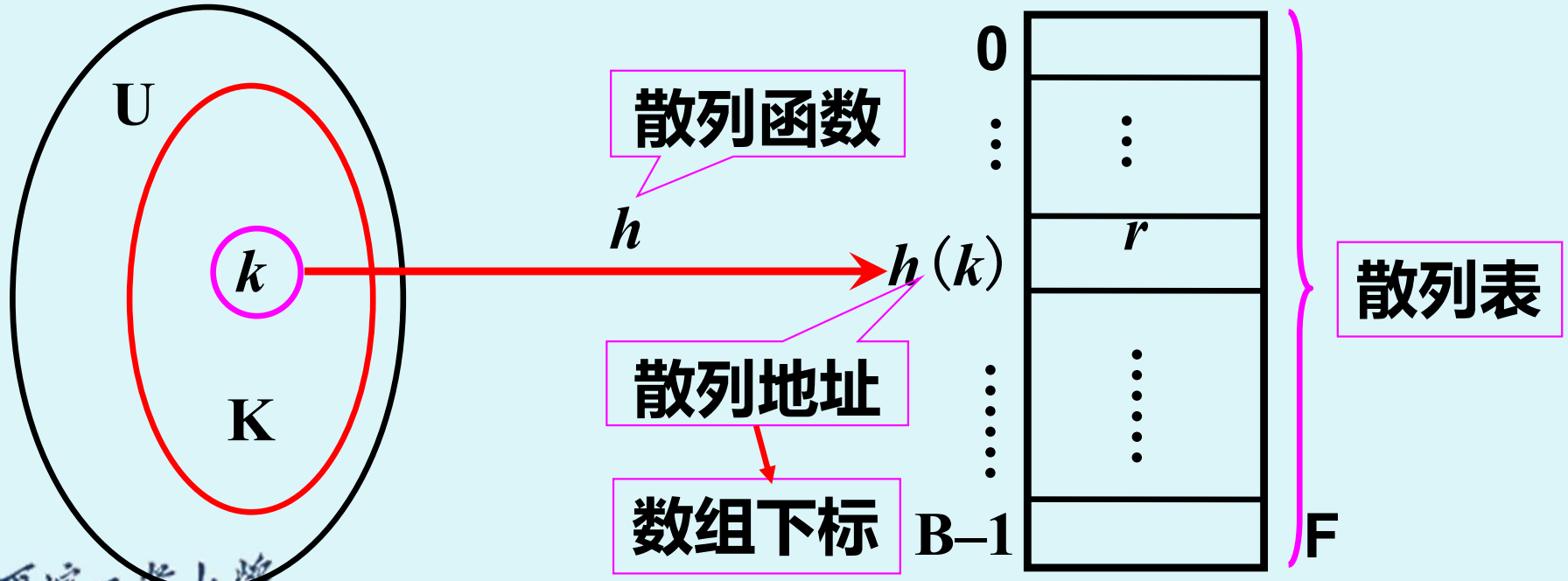
设 U 表示所有可能出现的关键字集合，K表示实际出现（实际存储）的关键字集合，即K ⊆U，F[B – 1]是一个数组，其中B =O(| K|)。则，

- 从 U 到表F[B – 1]下标集合上的一个映射 *h*：U—>{ 0, 1, 2, …,B –1}称为散列函数（哈希函数，杂凑函数）



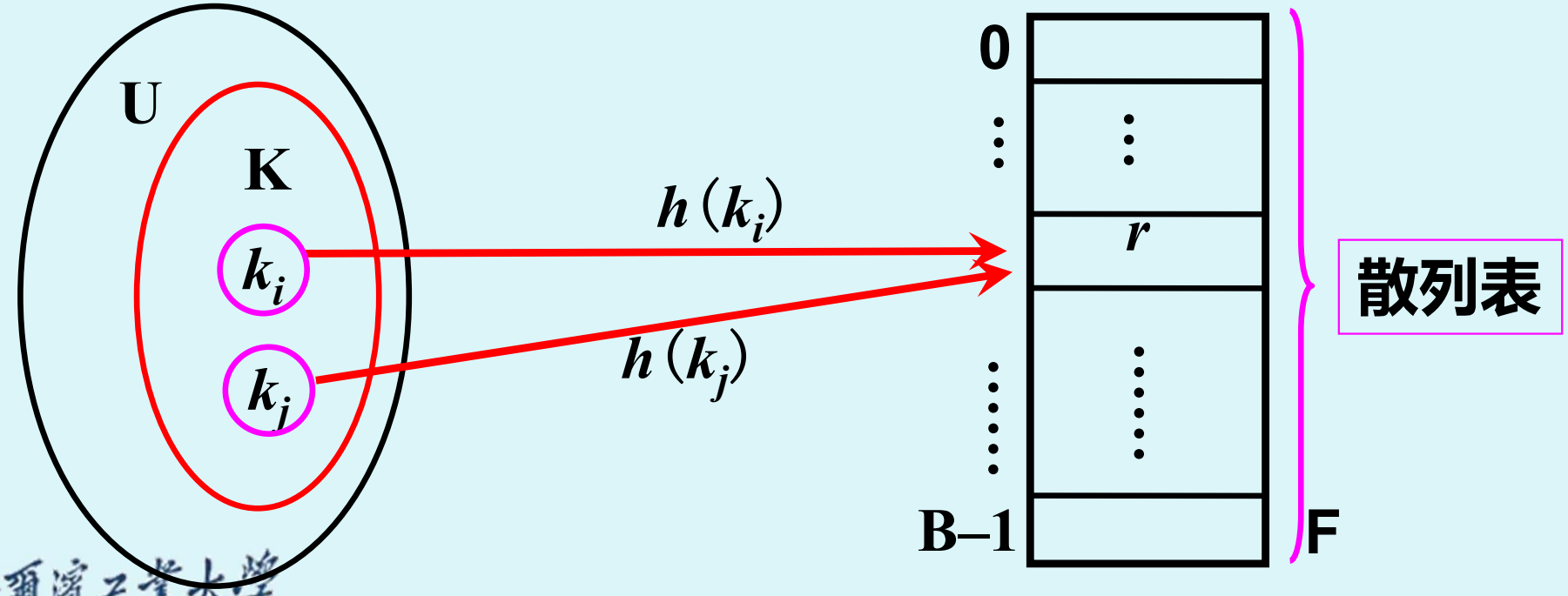
哈希（散列）技术

- 散列技术的相关概念
  - 数组 F 称为散列表（Hash表，杂凑表）。数组 F 中的每个单元称为桶(bucket)。
  - 对于任意关键字$k \in U$，函数值$h(k)$称为 k 的散列地址（Hash地址，散列值，存储地址，桶号）



哈希（散列）技术

- 散列技术的相关概念
  - 将结点（记录）按其关键字的散列地址存储到散列表中的过程称为散列。(collision)
  - 不同的关键字具有相同散列地址的现象称为散列冲突（碰撞）。而发生冲突的两个关键字称为同义词(synonym)。



5.8 散列技术（Cont.）

- 散列技术仅仅是一种查找技术吗？
  - 散列既是一种查找技术，也是一种存储技术。
- 散列是一种完整的存储结构吗？
  - 散列只是通过记录的关键字的值定位该记录，没有表达记录之间的逻辑关系，所以散列主要是面向查找的存储结构
- 散列技术适用于何种场合？
  - 通常用于实际出现的关键字的数目远小于关键字所有可能取值的数量。
- 散列技术适合于哪种类型的查找？
  - 不适用于允许多个记录有同样关键字值的情况。
  - 也不适用于范围查找，如在散列表中，找最大或最小关键值的记录，也不可能找到在某一范围内的记录。
- 散列技术最适合回答的问题是：如果有的话，哪个记录的关键字的值等于待查值。

哈希（散列）技术

- 散列技术需解决的关键问题：
  - 散列函数的构造。
    - 如何设计一个简单、均匀、存储利用率高的散列函数
  - 冲突的处理
    - 如何采取合适的处理冲突方法来解决冲突。
  - 散列结构上的查找、插入和删除

散列函数的构造

- 散列函数的构造的原则：
  - 计算简单：散列函数不应该有很大的计算量，否则会降低查找效率。
  - 分布均匀：散列函数值即散列地址，要尽量均匀分布在地址空间，这样才能保证存储空间的有效利用并减少冲突。

8.3.1 什么是哈希表

1. 哈希表的引入

例 学生名单的顺序存储与查找方式：

030530101　张三　男

030530103　李四　男

……

030530133　王五　男

030530101	030530103	…	030530133
张三	李四	…	王五
男	男	…	男
0	1	…	…

1. 哈希表的引入

030530101 张三 男
030530103 李四 男
……
030530133 王五 男

构造一个函数，对每一条记录，输入关键字，输出一个整数值。例如:构造函数H(x)如下:

H(x)=atoi(substr(x,8,2))-1

在存储时，把要存储的记录的关键字代入函数，把记录存入函数的返回值的位置。例如：存储“张三”时：把张三的学号代入函数H(x)=atoi(substr(“030530101”x,8,2))-1=0.

函数值为0，把张三的信息存入0下标位置

030530101		030530103	…	030530133	…
张三		李四	…	王五	…
男		男	…	男	…
0	1	2	…	32	…

2. 哈希表的思想

以结点的关键字K为自变量，通过一个确定的函数关系H，计算出对应的函数值H(K),把这个函数值作为结点的存储地址。

存储时：

把关键字为K的数据元素存储在地址H(K)中;

查找时：

给定关键字K，直接到地址H(k)中取数据元素。

这种方法称作散列法。

3. 哈希表的定义

用散列法存储的线性表叫散列表（Hash table）,又称杂凑表，哈希表，对应的函数称为散列函数、杂凑函数或哈希函数。

4. 哈希表的有关概念

(1) 装填因子：

设散列表空间大小为n，填入表中的结点数为m，则称α=m/n为散列表的装填因子。

(2) 散列函数的选取原则：

①运算简单；

②函数值域不能超过表长；

③尽可能使关键字不同时，散列函数值也不同。

(3) 冲突与同义词

设k1和k2是两个不同的关键字，若H(k1)=H(k2)，则称为冲突，

发生冲突的两个关键字k1和k2称为同义词。

第8章 查找

8.3.1 什么是哈希表

8.3.2 哈希函数的构造方法 ⟹

8.3.3 处理冲突的方法

8.3.4 哈希表的查找及其分析

1. 直接定址法
2. 质数除余法
3. 平方取中法
4. 折叠法
5. 数字分析法
6. 基数转换法

8.3.2 哈希函数的构造方法

1. 直接定址法

关键字：1510， 1620， 1930， 3050， 9060

H(key)=key/10-150

取关键字或关键字的某个线性函数值为哈希地址。

即：H(key)=a*key+b

其中a、b为常数。又称H(key)为自身函数。

优点：没有冲突；

缺点：若关键字集合取值跨度很大，浪费存储空间严重。

2. 质数除余法

关键字：15, 20, 19, 7, 10, 100, 230

例如：H(key)=key%11

如果表长为n，取小于或等于n的最大质数m作模，关键字通过m取模运算，所得值作为散列地址。

为什么用质数?

可以减少冲突。

8.3.2 哈希函数的构造方法

- 平方取中法
- 折叠法
- 数字分析法
- 基数转换法

第8章 查找

8.3.1 什么是哈希表

8.3.2 哈希函数的构造方法

8.3.3 处理冲突的方法

8.3.4 哈希表的查找及其分析

- 开放定址法
- 拉链法

1. 定义
2. 构造演示
3. 删除演示
4. 处理冲突的方法

8.3.3 处理冲突的方法

1. 开放定址法

冲突时在周围找一个新的空闲的地址。

8.3.3 处理冲突的方法

2. 开放定址法示例演示

例 设散列表的长度为13，散列函数为：H(K)=K%13,给定的关键字序列为：

19, 14, 23, 1, 68, 20, 84, 27, 54, 11, 10,

H(K)=6, 1, 10, 1, 3, 7, 6, 1, 2, 11, 10,

	14	1	68	27	54	19	20	84		23	11	10
0	1	2	3	4	5	6	7	8	9	10	11	12

(1)线性探测再散列法：若H(key)=d的单元发生冲突，则按下述方法进行探查：

$h_i(k)=(h(k)+i)\%n$,n是散列表的长度，$1\le i\le n-1$

二次聚集的概念

两个本来不发生冲突的非同义词，也就是散列地址不同的结点，发生争夺同一个散列地址的现象称为二次聚集或堆积。

- 同义词冲突：两个不同关键字记录的哈希函数值相同
- 非同义词冲突：多个不同哈希函数值的记录争抢同一地址
- 堆积现象：指非同义词冲突出现的现象，拉链法不会引起堆积现象。

8.3.3 处理冲突的方法

2. 开放定址法示例演示

19, 14, 23, 1, 68, 20, 84, 27, 54, 11, 10,

H(K)=6,　1, 10, 1, 3,　7,　6,　1,　2,　11, 10,

	14	1	68	27	54	19	20	84		23	11	10
0	1	2	3	4	5	6	7	8	9	10	11	12

查找分析：查11，查40。

1、利用散列函数计算出关键字为K的数据元素的地址：d=H(K)，如果F[d].key==K,查找成功，返回d;

2、如果F[d].key!=K,依次查找F[(d+i)%n].key, 直到找到某个F[(d+j)%n].key==K, 返回(d+j)%n, 或者找到一个开放地址为止，或者又回到F[d], 此时表示没找到。

3. 开放定址法删除示例演示

19, 14, 23, 1, 68, 20, 84, 27, 54, 11, 10,

H(K)=6, 1, 10, 1, 3, 7, 6, 1, 2, 11, 10,

	14	1	68	27	54	19	20	84		23	11	10
0	1	2	3	4	5	6	7	8	9	10	11	12

删除分析：如果删除68，查找27。

删除结点时不能简单地将被删结点的地址置空，否则将截断它之后填入散列表的同义词的查找路径。

删除时，设置特殊标志表示该位置元素被删除。

4 开放定址法处理冲突的方法

(1)线性探测再散列法：若H(key)=d的单元发生冲突，则按下述方法进行探查：

$h_i(k) = (h(k) + 1) \% n$, n是散列表的长度，$1 \le i \le n-1$

(2)二次探测再散列法：若H(key)=d的单元发生冲突，则按下述方法进行探查：

$h_i(k) = (h(k) + di) \% n$, n 是散列表的长度，

$di=1^2,-1^2,2^2,-2^2,\ldots,+k^2,-k^2,(k \le n/2)$

(3)di=伪随机序列，称伪随机探测再散列。

4 开放定址法处理冲突的方法

(4)再哈希法（再散列法）

设置$RH_i()$ $i=1,2,\ldots,k$, 多个哈希函数，当一个哈希函数产生冲突时，顺序用下一个。

第8章 查找

8.3.1 什么是哈希表

8.3.2 哈希函数的构造方法

8.3.3 处理冲突的方法

8.3.4 哈希表的查找及其分析

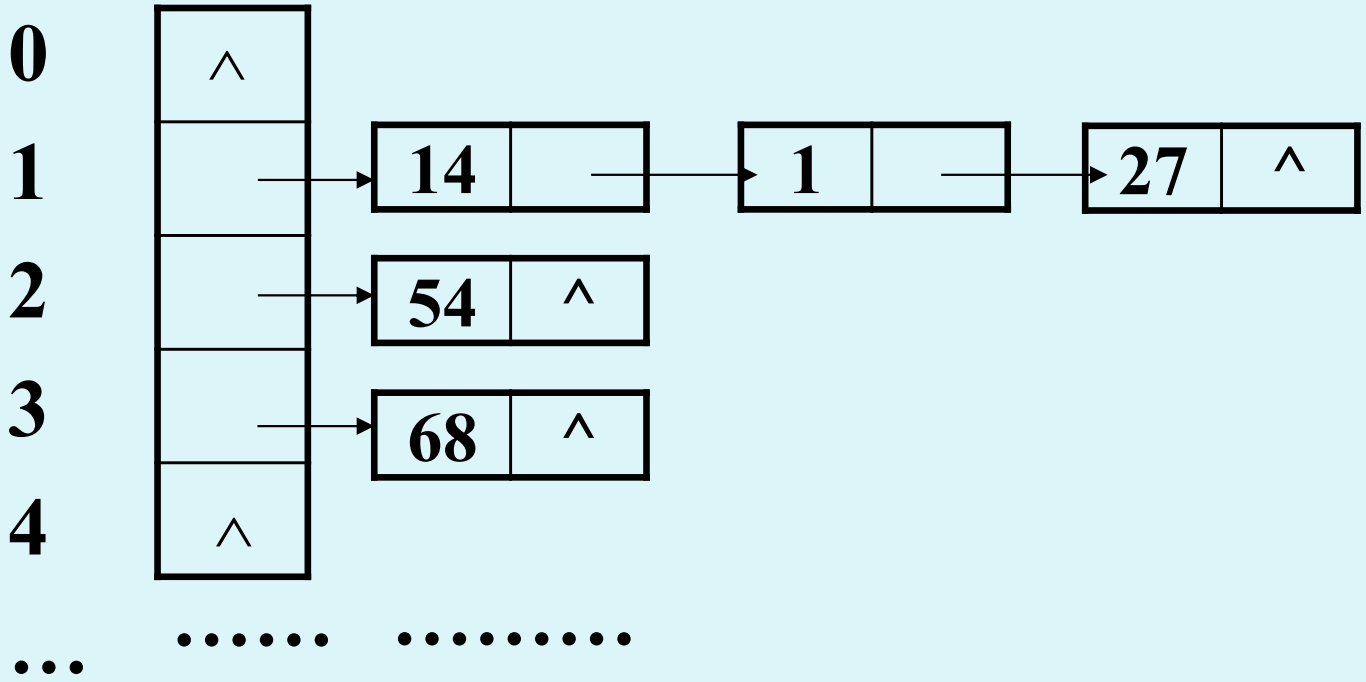
- 开放定址法
- 拉链法

1. 实例分析
2. 比较
3. 查找算法
4. 插入算法

1. 链地址法（又称拉链法）的示例演示

19, 14, 23, 1, 68, 20, 84, 27, 54, 11, 10,

H(K)=6, 1, 10, 1, 3, 7, 6, 1, 2, 11, 10,

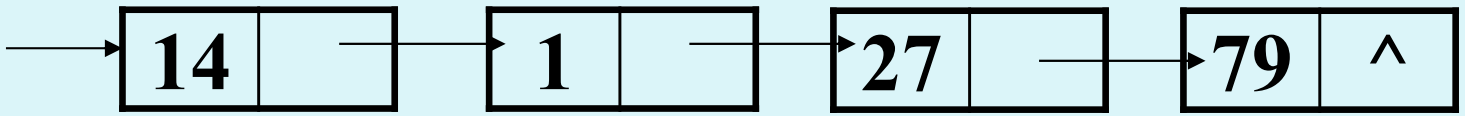


2. 开放定址法与链地址法平均查找长度比较

链地址法（拉链法，外散列表）平均查找长度短。

查79

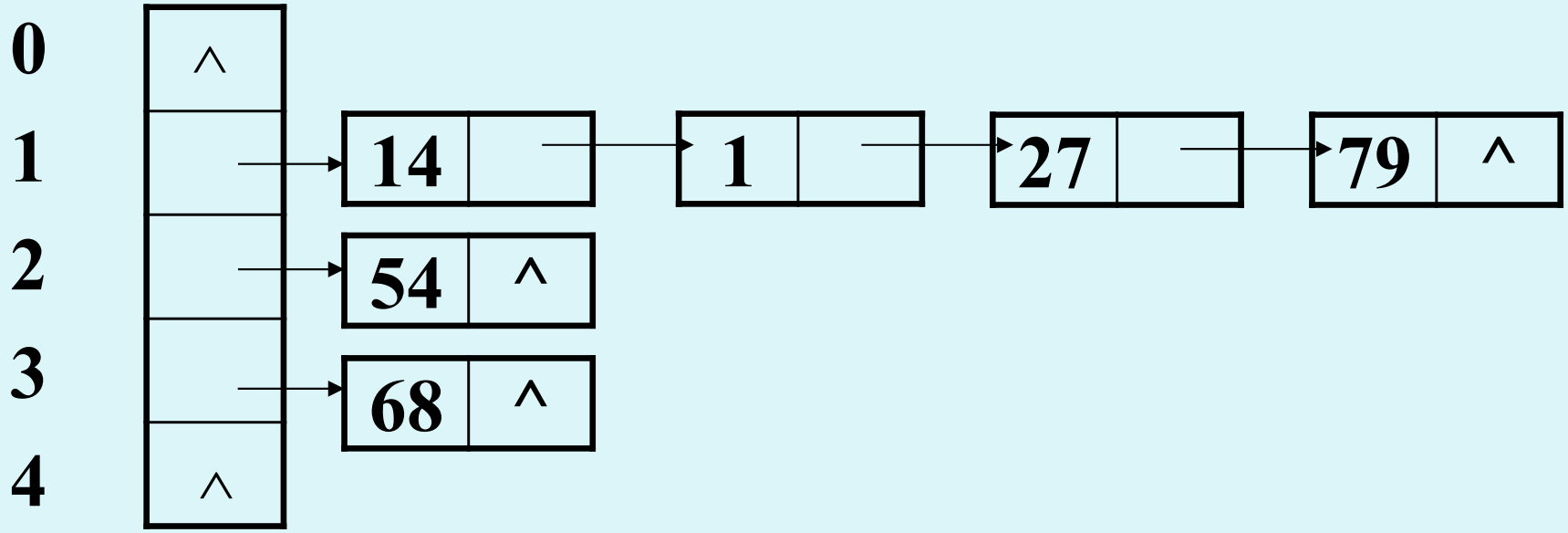
	14	1	68	27	54	19	20	84	79	23	11	10
0	1	2	3	4	5	6	7	8	9	10	11	12



3. 拉链法的查找算法

(1) 根据关键字K找到关键字为K的结点所在单链表的首地址；

(2)在所找到的单链表上进行顺序查找，若找到返回地址值，否则返回空值。



4. 拉链法的插入算法：

(1) 根据关键字K找到关键字为K的结点所应该存在的单链表的首地址；

(2)在单链表中查找结点为K的关键字，若找到，则无须插入；

(3)若没找到，利用头插法或尾插法插入单链表中。

单选题 1分

设置

以下关于哈希查找的叙述中错误的是（ ）。

A 哈希函数H(k)=k MOD p，p通常取小于等于表长的素数

B 用线性探测法解决冲突易引起堆积现象

C 用拉链法解决冲突易引起堆积现象

D 哈希函数选得好可以减少冲突现象

提交

第8章 查找

8.3.1 什么是哈希表

8.3.2 哈希函数的构造方法

8.3.3 处理冲突的方法

8.3.4 哈希表的查找及其分析

⇒

1. 存储结构
2. 查找算法
3. 算法分析
4. 与其他方法的比较

8.3.4 哈希表的查找和分析

1. 哈希表存储结构

```
int  hashsize[] = { 997, ... };
typedef struct {
  ElemType  *elem;
  int  count;          // 当前数据元素个数
  int  sizeindex;
            // hashsize[sizeindex]为当前容量
} HashTable;
#define  SUCCESS  1
#define  UNSUCCESS  0
#define  DUPLICATE  -1
```

2. 哈希表的查找算法

```
Status SearchHash (HashTable H, KeyType K, int &p, int &c)
{   p = Hash(K);        // 求得哈希地址

    while ( H.elem[p].key != NULLKEY &&
                    !EQ(K, H.elem[p].key))
        collision(p, ++c);      // 求得下一探查地址 p

    if ( EQ(K, H.elem[p].key) ) return SUCCESS;
            // 查找成功

    else return UNSUCCESS;   // 查找不成功
} // SearchHash
```

哈希表查找的性能分析

- 由于冲突的存在，产生冲突后的查找仍然是给定值与关键码进行比较的过程。
- 在查找过程中，关键码的比较次数取决于产生冲突的概率。而影响冲突产生的因素有：
  - 散列函数是否均匀
  - 处理冲突的方法
  - 散列表的装载因子

    α=表中填入的记录数/表的长度

哈希表查找的性能分析

- 几种不同处理冲突方法的平均查找长度

ASL / 处理冲突方法	查找成功时	查找不成功时
线性探测法	$\frac{1}{2}(1+\frac{1}{(1-\alpha)^2})$	$\frac{1}{2}(1+\frac{1}{1-\alpha^2})$
二次探测法	$-\frac{1}{\alpha}\ln(1+\alpha)$	$\frac{1}{1-\alpha}$
拉链法	$1+\frac{\alpha}{2}$	$\alpha+e^{-\alpha}$

哈希表的平均查找长度是 α 的函数，而不是 n 的函数。

这说明，用哈希表构造查找表时，可以选择一个适当的装填因子 α，使得平均查找长度限定在某个范围内。

8.3.4 哈希表的查找和分析

4. 散列法与其它方法的比较

(1) 不同的散列函数构成的哈希表平均查找长度是不同的。

(2) 由同一个散列函数、不同的**解决冲突方法**构造的哈希表，其平均查找长度是不相同的。

(3) 散列法与其他查找方法的区别:

除散列法外，其他查找方法有共同特征为：均是建立在比较关键字的基础上。

散列法是根据关键字直接求出地址的查找方法，其查找的期望时间为O(1)。

单选题 1分

设置

为提高哈希（Hash）表的查找效率，可以采取的正确措施是（ ）。

Ⅰ.增大装填（载）因子

Ⅱ.设计冲突（碰撞）少的哈希函数

Ⅲ.处理冲突（碰撞）时避免产生堆积（堆积）现象

A 仅Ⅰ

B 仅Ⅱ

C 仅Ⅰ、Ⅱ

D 仅Ⅱ、Ⅲ

提交

单选题 1分

设置

以下关于哈希查找的叙述中正确的是（ ）。

A 哈希表在查找成功时的平均查找长度仅仅与表长有关

B 哈希表的装填因子等于表中填入的记录数除以哈希表的长度

C 哈希查找中不需要任何关键字的比较

D 采用拉链法解决冲突时，查找一个元素的时间是相同的

提交

主观题 10分

设置

选取哈希函数H(k)=(3k) MOD 11。用开放定址法处理冲突,di =i ((7k) MOD 10+1)(i=1,2,3..)。试在0~ 10的散列地址空间中对关键字序列(22,41,53,46,30,13,01,67) 构造哈希表，并求等概率情况下查找成功时的平均查找长度。

H(22)=0;
H(41)=2;
H(53)=5;
H(46)=6;
H(30)=2; -> H(30)=3;
H(13)=6; -> H(13)=8;
H(01)=3; -> H(01)=0; -> H(01)=8; -> H(01)=5; -> H(01)=2; -> H(01)=10;
H(67)=3; -> H(67)=2; -> H(67)=1;

0	1	2	3	4	5	6	7	8	9	10
22	67	41	30		53	46		13		01

等概率时查找成功的平均查找长度为:

ASL = (1+3+1+2+1+1+2+6)/8=17/8

本章小结

(1) 熟练掌握顺序查找、二分查找、二叉排序树查找以及散列表查找的基本思想和算法实现。

(2) 熟练掌握顺序查找、二分查找、二叉排序树查找以及散列表查找的应用条件以及算法优劣。

(3) 掌握二叉排序树的删除算法和平衡二叉树的构造算法的概要。

复　习

一、　选择题

1.若查找每个记录的概率均等，则在具有n个记录的连续顺序文件中采用顺序查找法查找一个记录，其平均查找长度ASL为(C)。

A.　(n-1)/2　B. n/2

C.　(n+1)/2　D. n

复　习

2. 下面关于二分查找的叙述正确的是（　）D

A.表必须有序，表可以顺序方式存储，也可以链表方式存储

C.表必须有序，而且只能从小到大排列

B.表必须有序且表中数据必须是整型，实型或字符型

D.表必须有序，且表只能以顺序方式存储

复　习

3．当在一个有序的顺序存储表上查找一个数据时，即可用折半查找，也可用顺序查找，但前者比后者的查找速度(　　)C

A.必定快 B.不一定

C.在大部分情况下要快

D.取决于表递增还是递减

复　习

4．二叉排序树的查找效率与二叉树的(c(1))有关,在 (c(2))时其查找效率最低

(1):A.高度　B.结点的多少
　C. 树型　D.结点的位置
(2):A.结点太多　B. 完全二叉树
　C.呈单枝树 D. 结点太复杂。

复　习

5. 设有一组记录的关键字为{19，14，23，1，68，20，84，27，55，11，10，79}，用链地址法构造散列表，散列函数为H（key）=key MOD 13,散列地址为1的链中有（　）个记录。

A. 1　　B. 2　　D

C. 3　　D. 4

复　习

6．下面关于哈希(Hash，杂凑)查找的说法正确的是（　C）

A．哈希函数构造的越复杂越好，因为这样随机性好，冲突小

B．除留余数法是所有哈希函数中最好的

C．不存在特别好与坏的哈希函数，要视情况而定

D．若需在哈希表中删去一个元素，不管用何种方法解决冲突都只要简单的将该元素删去即可

复　习

7． 若采用链地址法构造散列表，散列函数为H（key）=key MOD 17，则需（A(1)）个链表。这些链的链首指针构成一个指针数组，数组的下标范围为（（2)C）

(1) A. 17　　B. 13
　　C. 16　　D. 任意

(2) A. 0至17　　B. 1至17
　　C. 0至16　　D. 1至16

复　习

8．　假定有k个关键字互为同义词，若用线性探测法把这k个关键字存入散列表中，至少要进行多少次探测?（ D）

A．k-1次　　B. k次

C. k+1次　　D. k（k+1）/2次

复　习

二、判断题

1．Hash表的平均查找长度与处理冲突的方法无关。

×

2.负载因子 (装填因子)是散列表的一个重要参数，它反映散列表的装满程度。

√

复　习

3. 对一棵二叉排序树按前序方法遍历得出的结点序列是从小到大的序列。

×

4. 在任意一棵非空二叉排序树中，删除某结点后又将其插入，则所得二叉排序树与原二叉排序叉树相同。

×

复　习

三、填空题

1.动态查找表和静态查找表的重要区别在于前者包含有__插入__和__删除__运算，而后者不包含这两种运算。

2.顺序查找n个元素的顺序表，若查找成功，则比较关键字的次数最多为__n__次；当使用监视哨时，若查找失败，则比较关键字的次数为__n+1__。

复　习

3.分块检索中，若索引表和各块内均用顺序查找，则有900个元素的线性表分成 30 块最好：若分成25块，其平均查找长度为 31.5。

二分查找的递归算法

```
int low=0,up=last;
int BINSEARCH(int low,int up,LIST F)
{ int mid;
  if(low<=up)
 {mid=(low+up)/2;
  if (K==F[mid].key) return mid;
  else
  if (K<F[mid].key)
   retrun BINSEARCH(low,mid-1,F)
   else
   return BINSEARCH(mid-1,up,F);
  }
  return(-1);
}
```

***

已知二叉树中的结点类型BinTreeNode定义为：

```
struct BinTreeNode{
ElemType data;
BinTreeNode*left，*right};
```

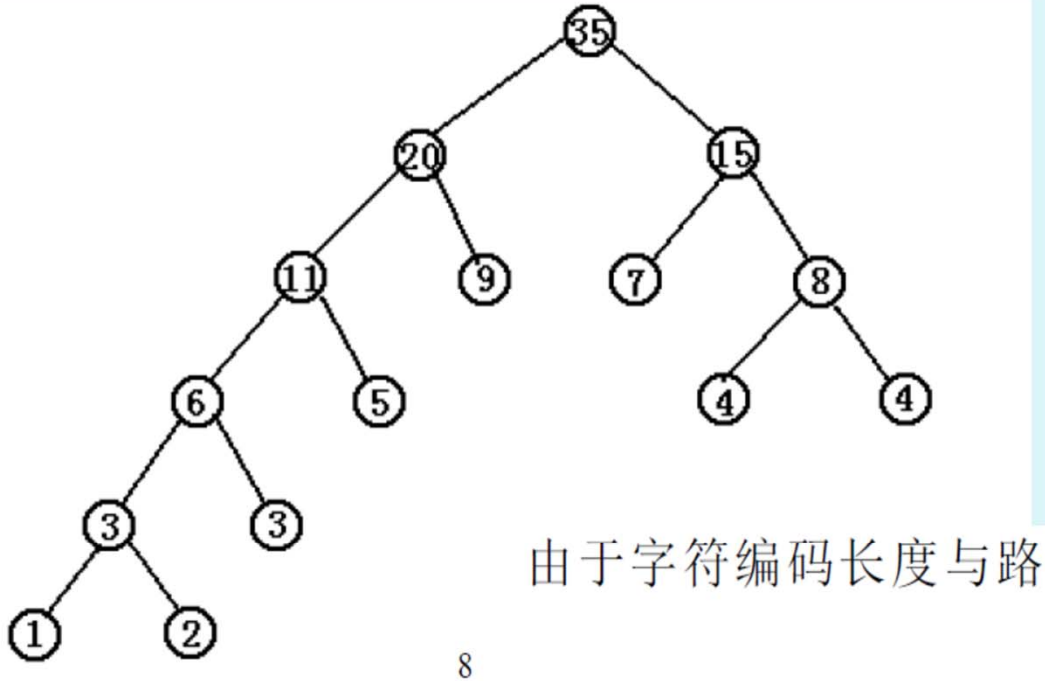
其中data为结点值域，left和right分别为指向左、右子女结点的指针域。

下面函数的功能是返回二叉树BT中值为x的结点所在的层号，请补全最内层else所包含的内容.

```
Int NodeLevel(BinTreeNode * BT, ElemType X)
{
  if(BT==NULL)return 0;//空树的层号为0
  else if(BT->data==X)
            return 1;//根结点的层号为1
        else{//向子树中查找x结点
              int c1=NodeLevel(BT->left, X);
              if(c1>=1)return c1+1;
              int c2=NodeLevel(BT->right, X);
              if (c2>=1)returnc2 + 1
              //若树中不存在X结点则返回0
               else return 0;
            }
  }
```

已知某字符串S中共有8种字符，各种字符分别出现2次、1次、4次、5次、7次、3次、4次和9次，对该字符串用｛0，1｝进行前缀编码，问该字符串的编码至少有多少位？

按照哈夫曼树的算法，以字符出现次数作为权，可构造出8种字符构造的哈夫曼树，如下图所示。



由于字符编码长度与路径长度相等，所以字符串的编码至少有：

$$\sum_{i=1}^{8} w_i l_i = 5*1+5*2+4*3+3*5+3*4+3*4+2*9+2*7=98 \text{ 位。}$$